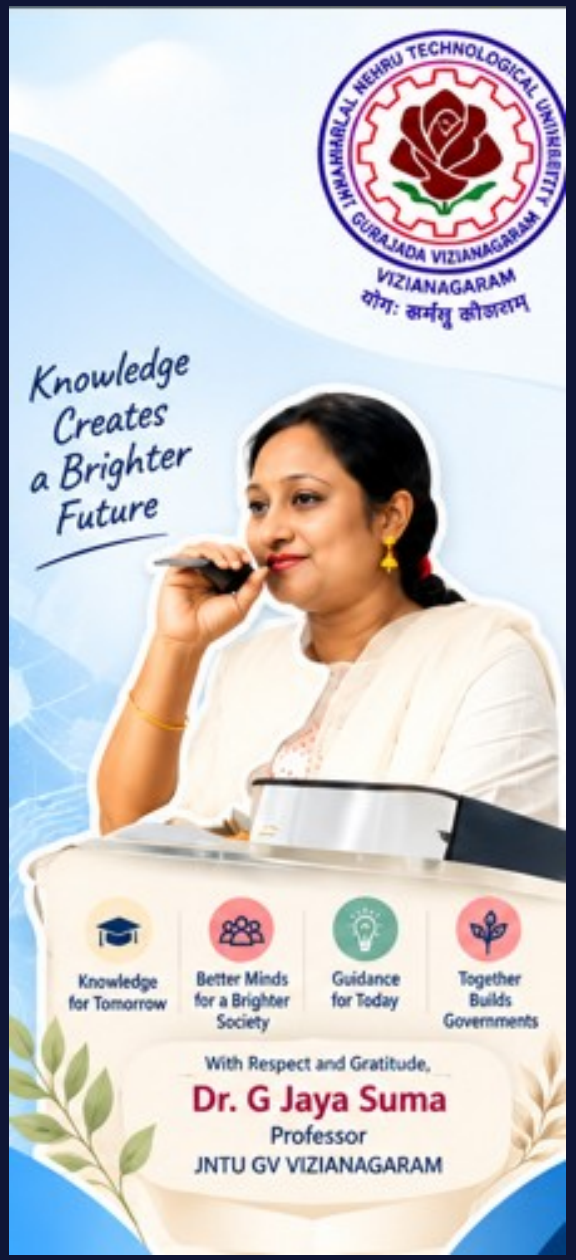


JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

What Makes an Algorithm "Quantum"?

The ingredients that set quantum algorithms apart: superposition, interference and entanglement.

15 MIN

Amplitudes | Interference | Entanglement | Measure

Fundamentals of Quantum Computing · Quantum Algorithms: Introduction

1 What we'll cover in 15 minutes

Four short segments, from circuits we can build to what makes an algorithm quantum.

1. **Recap: gates and circuits** — 3 min
   We can build and simulate circuits — now we ask what to compute with them.
2. **The recipe of a quantum algorithm** — 4 min
   Superposition, a problem gate, interference, then measurement.
3. **Worked example: interference** — 4 min
   Two H gates show amplitudes cancelling — impossible for probabilities.
4. **What is different from classical** — 4 min
   Classical and quantum ingredients, side by side.

2 Recap: gates and circuits

We can build and simulate circuits — the next question is what to compute with them.

In simple words

So far we have written gates, chained them into circuits, and checked them on a simulator. An algorithm is a circuit designed to solve a problem.

Circuit *+ goal* **Algorithm**

An algorithm is a circuit plus a purpose: a problem to solve and a way to read the answer.

G

Gates

Unitary building blocks: X, H, T, CNOT and their relatives.

C

Circuits

Gates in order on a register of qubits, ending in measurement.

?

The missing piece

Which circuits solve useful problems — and why might they beat classical ones?

3 The recipe of a quantum algorithm

Create superposition, apply the problem, then use interference to reveal the answer.

The standard pattern

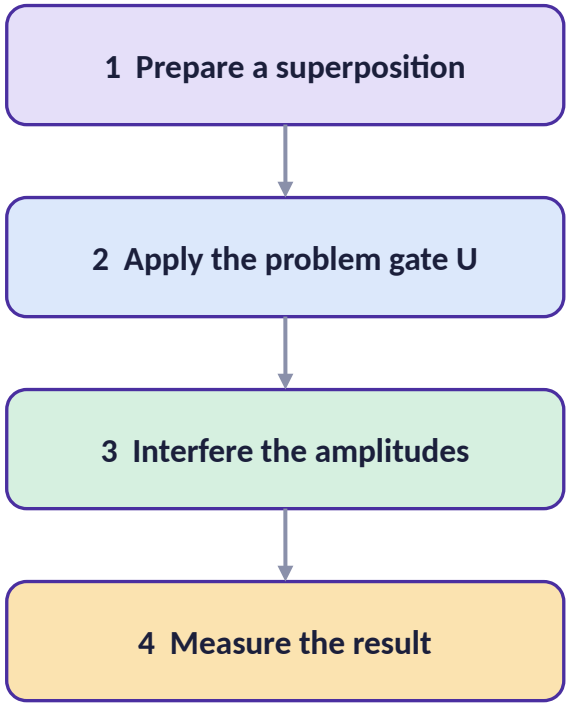
$$|0\ldots0\rangle \rightarrow \mathrm{H} \rightarrow \mathrm{U} \rightarrow \mathrm{H} \rightarrow \text{measure}$$

Many algorithms **create a superposition**, apply the problem as a gate U, then let interference concentrate probability on the answer.

Not: trying every answer and reading them all. **Instead:** shaping amplitudes so the right answer is likely.

Key idea: a measurement returns only one outcome, so the algorithm must make the right one likely.

The four stages



Many quantum algorithms follow this pattern; the problem gate is what changes.

4 Worked example: interference

Two H gates show amplitudes cancelling — something plain probabilities can never do.

Example: H twice on $|0\rangle$

$$|0\rangle \rightarrow \mathrm{H} \rightarrow \mathrm{H} \rightarrow |0\rangle$$

A fair classical coin flipped twice stays 50/50, but H twice returns $|0\rangle$ every time: the $|1\rangle$ amplitudes cancel.



P(0) = 1 P(1) = 0

+

Constructive

Amplitudes with the same sign add up, making an outcome more likely.

−

Destructive

Amplitudes with opposite signs cancel, making an outcome less likely — even impossible.

%

Not just randomness

A classical random process has only probabilities, so it can never cancel like this.

5 What is different from classical

The same words — state, randomness, answer — take on new meanings.

Ingredient	Classical algorithm	Quantum algorithm
State	Bits: one definite pattern at a time	Qubits: amplitudes over all patterns
Randomness	Probabilities only	Amplitudes that can interfere
Correlations	Shared or copied data	Entanglement between qubits
Reading the answer	Look at any time	Measure — usually once, at the end
Source of advantage	A faster procedure or machine	Interference steering amplitudes to the answer

ONE-LINE TAKEAWAY

A quantum algorithm shapes amplitudes with superposition, entanglement and interference so that measuring gives the right answer with high probability.

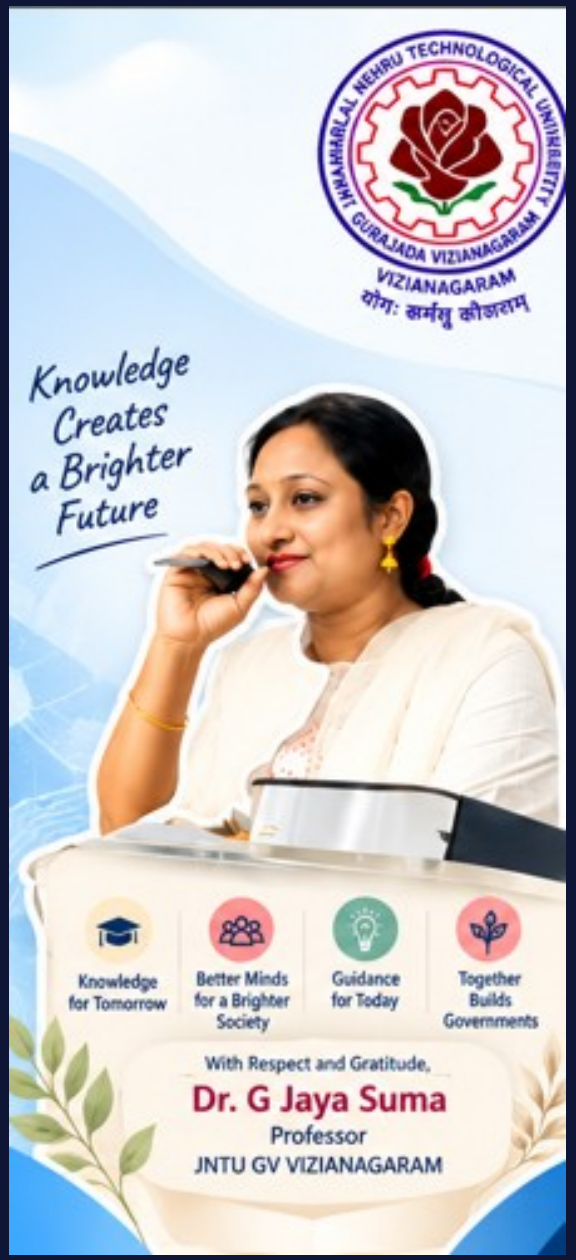
Superposition: many amplitudes

Interference: steer probability

Measurement: one answer

Next: Your First Quantum Algorithm

We apply this recipe to a small problem and watch interference find the answer.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Oracles and Quantum Parallelism

Black-box functions as gates, and how one call can touch every input at once.

35 MIN

Oracle | Parallelism | Phase kickback | Readout

1 What we'll cover in 35 minutes

Five segments, from the oracle idea to what one run can actually read out.

1. **Recap: the algorithm recipe** — 5 min
   Stage 2 of the recipe — the problem gate — is usually an oracle.
2. **Oracles: a function as a gate** — 8 min
   Turning f(x) into a reversible, unitary black box.
3. **Worked example: quantum parallelism** — 7 min
   One call evaluates f on both inputs at once.
4. **Oracle types at a glance** — 8 min
   Bit-flip, phase, kickback, constant and marking oracles.
5. **Phase kickback and the readout limit** — 7 min
   How oracles write phases, and why one run reads only one result.

2 Recap: the algorithm recipe

Prepare, apply the problem gate, interfere, measure — today we open up stage 2.

In simple words

The problem gate in our recipe is usually an oracle: a black box that computes a function f. We can call it, but we cannot look inside.



We may only call the oracle — we cannot read how it computes f.

f

A function f

Maps each input x to an output, such as a single bit.

U

Made unitary

The oracle must be reversible so it can run inside a quantum circuit.

?

A black box

The algorithm learns about f only by calling the oracle.

3 Oracles: a function as a gate

To run f on a quantum computer, wrap it in a reversible, unitary gate.

The standard oracle

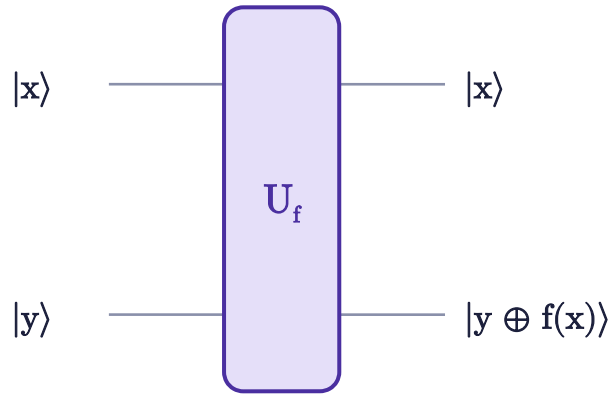
$$U_f\,|x,\,y\rangle = |x,\,y \oplus f(x)\rangle$$

The input **x is kept**, and f(x) is XORed into a spare qubit y — nothing is erased, so the oracle is reversible.

Start with y = 0: the spare qubit then holds f(x) exactly. **Apply it twice:** it undoes itself.

Why keep x? f alone may lose information; keeping x makes the whole map reversible.

The oracle as a circuit



The top wire passes through unchanged; the bottom wire gets f(x) XORed in.

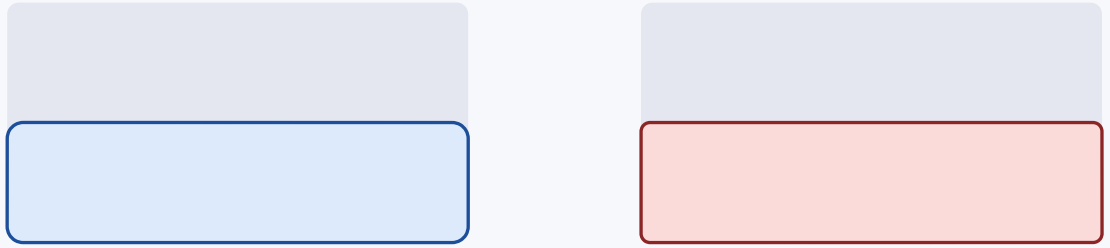
4 Worked example: quantum parallelism

One call evaluates f on both inputs at once — but only one result can be read.

Example: f(x) = 1 ⊕ x, input |+⟩ and y = 0

$$U_f|+\rangle|0\rangle = 0.707|0,1\rangle + 0.707|1,0\rangle$$

After a single call the state holds f(0) = 1 and f(1) = 0 together — but a measurement returns just one pair.



P(0,1) = 0.5 P(1,0) = 0.5

2

Two values, one call

A single use of the oracle carries both f(0) and f(1) in the state.

1

One pair per run

Measuring gives a single pair (x, f(x)) — here 50% each.

!

Parallelism is not enough

We need interference to turn a hidden pattern into a likely outcome.

5 Oracle types at a glance

The same function f can be wrapped in different kinds of oracle.

Oracle	Rule	Effect
Bit-flip oracle	$\|x, y\rangle \to \|x, y \oplus f(x)\rangle$	Writes f(x) into a spare qubit
Phase oracle	$\|x\rangle \to (-1)^{f(x)} \|x\rangle$	Marks inputs with a sign; no spare qubit needed
Constant oracle	f(x) is the same for every x	As a phase oracle: only a global sign, no physical effect
Marking oracle	f(x) = 1 only for one item w	Flips the sign of $\|w\rangle$ and nothing else
Phase kickback	Target $\|-\rangle$ turns a bit-flip oracle into a phase one	Same gate, different target state

6 Phase kickback

With the target in $|-\rangle$, the oracle writes f(x) into a sign on the input.

Input $|x\rangle$, target $|-\rangle$

$$U_f\,|x\rangle|-\rangle = (-1)^{f(x)}\,|x\rangle|-\rangle$$

Bit-flipping $|-\rangle$ only changes its sign, and that sign is "kicked back" onto the input register.

↓ *Apply the oracle*

The sign depends on f(x)

$+|x\rangle$

when f(x) = 0

$-|x\rangle$

when f(x) = 1

The target $|-\rangle$ ends the same way it began.

Target unchanged

X only multiplies $|-\rangle$ by −1, so the target finishes as it started.

Sign lands on the input

The −1 phase kicks back onto $|x\rangle$ — a phase oracle with no spare qubit left over.

Interference reads it

A final layer of H gates turns a pattern of signs into measurable probabilities.

7 Parallelism grows fast — readout does not

n input qubits hold 2^n values of x at once, but each run returns just one pair.

One call to the oracle on n input qubits in equal superposition

n = 1	n = 2	n = 3	n = 10	n = 20
Values of x in one call	Values of x in one call	Values of x in one call	Values of x in one call	Values of x in one call
2 inputs	4 inputs	8 inputs	1,024 inputs	~1 million inputs
Read out per run	Read out per run	Read out per run	Read out per run	Read out per run
1 pair (x, f(x))	1 pair (x, f(x))	1 pair (x, f(x))	1 pair (x, f(x))	1 pair (x, f(x))

The takeaway for this segment: Quantum parallelism gives access to all 2^n values of f at once, but only interference can turn that into a useful answer.

ONE-LINE TAKEAWAY

An oracle turns a function into a unitary gate; parallelism evaluates it on every input at once — and interference decides what we can read out.

Oracle: f as a unitary

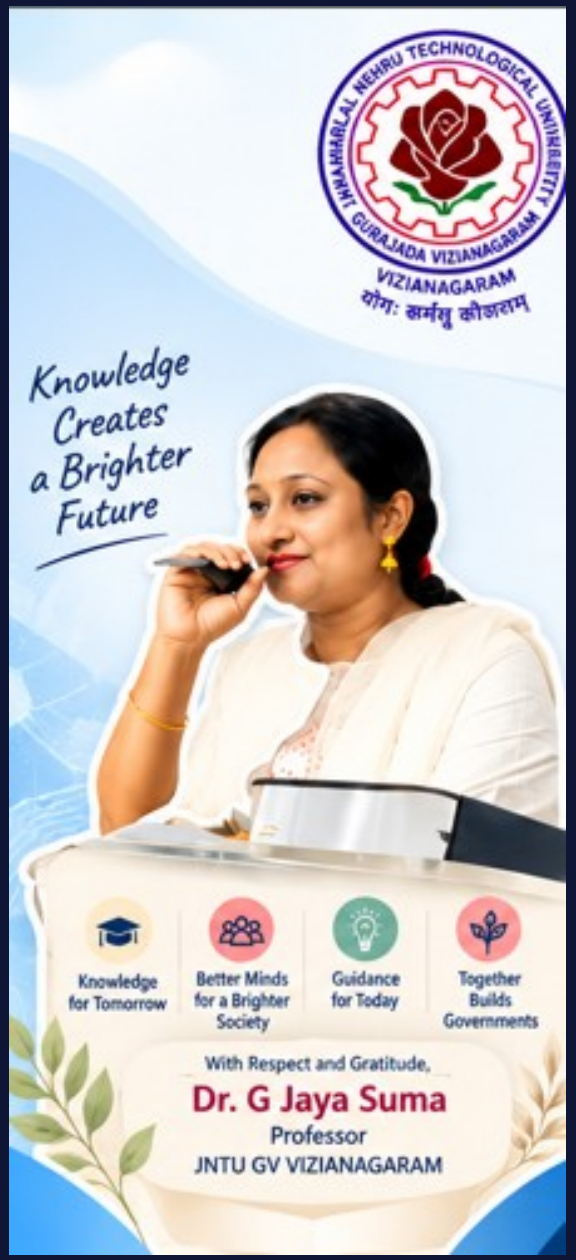
One call, one readout

Phase kickback

→

Next: Deutsch–Jozsa Algorithm

We combine an oracle, parallelism and interference to solve our first problem with fewer queries.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Deutsch and Deutsch–Jozsa Algorithms

Your first quantum speed-up: deciding constant or balanced with a single oracle call.

45 MIN

Deutsch | Deutsch–Jozsa | One query | Balanced

Fundamentals of Quantum Computing · Deutsch & Deutsch–Jozsa

1 What we'll cover in 45 minutes

Five segments, from one bit and one query to n bits and one query.

1. **Recap: oracles and phase kickback** — 5 min
   A function as a gate, and signs written onto the input.
2. **The Deutsch problem and circuit** — 10 min
   Decide constant or balanced for one bit with a single oracle call.
3. **Worked example: a balanced function** — 10 min
   Trace the circuit for f(x) = x and see why the answer is 1.
4. **All four one-bit functions** — 10 min
   Constant gives 0, balanced gives 1 — every time.
5. **Deutsch–Jozsa: from one bit to n bits** — 10 min
   The same circuit decides constant or balanced for any n.

2 Recap: oracles and phase kickback

Two ideas from the last topic combine to make today's algorithms.

In simple words

An oracle writes f(x) into a spare qubit. Prepare that qubit as $|-\rangle$ and the same gate instead writes f(x) into a sign on the input.



A sign is invisible to a measurement — but interference can turn it into a probability.

U

The oracle

Wraps a function f in a reversible, unitary gate.

−

Phase kickback

A target in $|-\rangle$ turns the bit-flip into a sign on the input.

H

Interference

H gates before and after convert signs into measurable results.

3 The Deutsch problem and circuit

Decide whether f is constant or balanced — with a single oracle call.

The problem

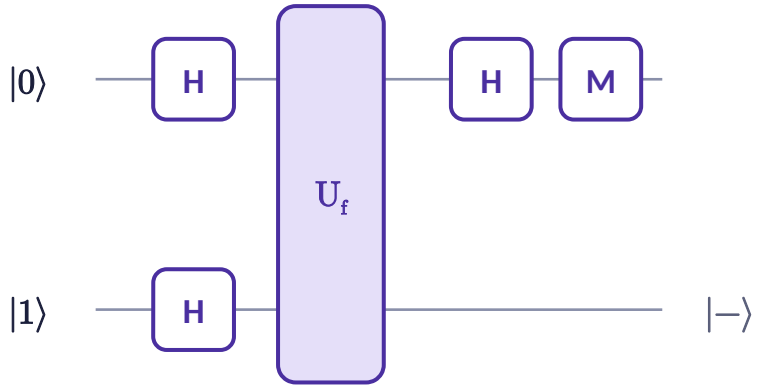
$$f(0) = f(1) \quad \text{or} \quad f(0) \neq f(1)\ ?$$

Given an oracle for f: {0,1} → {0,1}, decide if f is **constant or balanced** — using as few oracle calls as possible.

Classical: needs 2 calls, one for f(0) and one for f(1). **Quantum:** needs just 1 call.

Note: the circuit finds how f(0) and f(1) relate, not the two values themselves.

The Deutsch circuit



H on both qubits, one oracle call, H on the top qubit, then measure it: 0 means constant, 1 means balanced.

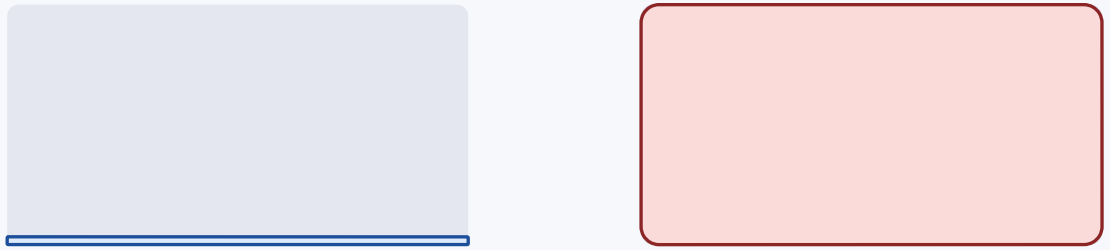
4 Worked example: a balanced function

Trace the circuit for f(x) = x and see why the answer is 1.

Example: f(x) = x, which is balanced

$$|0\rangle|1\rangle \rightarrow |+\rangle|-\rangle \rightarrow |-\rangle|-\rangle \rightarrow |1\rangle|-\rangle$$

H gives $|+\rangle|-\rangle$. The oracle turns the top qubit into $(|0\rangle - |1\rangle)/\sqrt{2} = |-\rangle$, and the last H sends $|-\rangle$ to $|1\rangle$.



0

Constant gives 0

If f(0) = f(1) the signs match, so the top qubit is $\pm|+\rangle$ and H returns 0.

1

Balanced gives 1

If f(0) ≠ f(1) the signs differ, so the top qubit is $\pm|-\rangle$ and H gives 1.

✓

Certain, not just likely

The answer is right with probability 1 — every run, after a single call.

5 All four one-bit functions

Every possible f is either constant or balanced, and the circuit tells them apart.

Function f	Type	Top qubit before the last H, and result
f(x) = 0	Constant	$\|+\rangle$, so H gives 0
f(x) = 1	Constant	$-\|+\rangle$, so H gives 0
f(x) = x	Balanced	$\|-\rangle$, so H gives 1
f(x) = 1 $\oplus$ x	Balanced	$-\|-\rangle$, so H gives 1
Oracle calls needed	Classical: 2 in the worst case	Quantum: just 1

6 Deutsch–Jozsa: from one bit to n bits

The same circuit works for an n-bit input — H on every input qubit, before and after.

The promise

$$f: \{0,1\}^n \to \{0,1\}$$

f is constant, or balanced: 0 for exactly half of the inputs and 1 for the other half.

$H \cdot U \cdot H$

Measure all n input qubits

0…0

all zeros: f is constant

anything else

f is balanced

One oracle call decides it, however large n is.

Same circuit

Replace each H by n parallel H gates; the structure stays identical.

One call

A single oracle call separates constant from balanced for any n.

Interference does the work

For a balanced f, the amplitude of $|0\ldots0\rangle$ cancels exactly.

7 Queries needed: classical vs quantum

A deterministic classical algorithm needs $2^{n-1} + 1$ queries in the worst case; the quantum one needs 1.

Worst case for a deterministic classical algorithm

n = 1	n = 2	n = 3	n = 10	n = 20
Classical queries	Classical queries	Classical queries	Classical queries	Classical queries
2	3	5	513	524,289
Quantum queries	Quantum queries	Quantum queries	Quantum queries	Quantum queries
1	1	1	1	1

The takeaway for this segment: The gap is exponential in queries — but only for this promise problem, and a randomized classical algorithm answers with high probability using just a few queries.

ONE-LINE TAKEAWAY

One oracle call plus interference decides whether f is constant or balanced — exponentially fewer queries than a deterministic classical algorithm.

Constant → all zeros

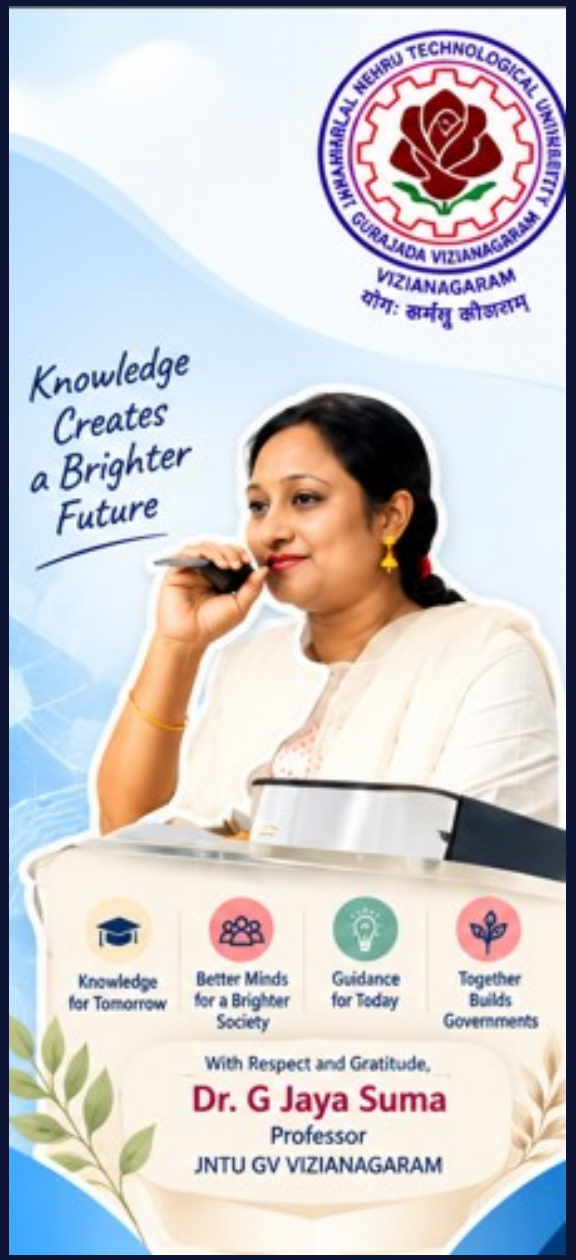
Balanced → not all zeros

One query, certain answer

→

Next: Grover's Search Algorithm

We use oracles and interference again — this time to find a marked item in an unsorted list.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

The Quantum Fourier Transform

From the discrete Fourier transform to phase estimation: the engine behind many quantum speed-ups.

45 MIN

QFT | Inverse QFT | Phase estimation | $O(n^2)$ gates

1

What we'll cover in 45 minutes

Six segments, from the classical DFT to the algorithm that puts the QFT to work.

1	**Discrete Fourier transform recap** Why phase estimation needs a Fourier transform on a quantum register.	5 min
2	**The QFT: definition and circuit** Action on basis states, then Hadamards and controlled-phase rotations.	12 min
3	**Worked example: a 2-qubit QFT** Follow $\|01\rangle$ through the circuit and check it against the QFT matrix.	8 min
4	**The inverse QFT** Run the circuit backwards with every rotation reversed.	5 min
5	**Quantum phase estimation** The circuit, and how to read the phase from the measurement.	10 min
6	**Complexity: QFT vs classical FFT** $O(n^2)$ gates against $O(N \log N)$ operations.	5 min

1

Recap: the discrete Fourier transform

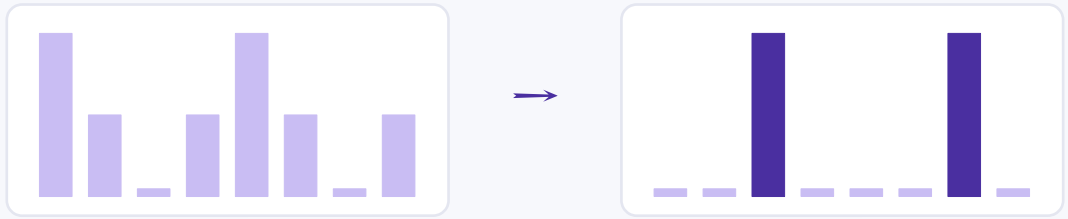
A Fourier transform turns a pattern into its frequencies, and phase estimation needs exactly that.

Discrete Fourier transform (DFT)

$$y_k = (1/\sqrt{N}) \sum x_j \, \omega^{jk}$$

with $\omega = e^{2\pi i/N}$, the N-th root of unity

The DFT re-expresses N numbers as N frequencies: a repeating pattern becomes a sharp peak.



signal x: a repeating pattern

DFT y: peaks at the frequency

Classical cost: the FFT computes the DFT in O(N log N) operations.

φ

Phase estimation stores φ as a wave

The counting register ends up in $(1/\sqrt{N}) \Sigma \, e^{2\pi i \varphi k} |k\rangle$: a frequency φ spread over the basis states.

F

A Fourier transform turns the wave into a peak

Applied to that register, the transform sends frequency φ to a single basis state that we can measure.

Q

We need it on qubits

Our data are amplitudes, not a list we can loop over, so we need a quantum circuit for it: the QFT.

2

The QFT: definition and action

The QFT is the DFT applied to the amplitudes of a quantum state.

Definition

$$\mathrm{QFT}\,|j\rangle = (1/\sqrt{N}) \sum e^{2\pi i \cdot jk/N}\,|k\rangle$$

sum over $k = 0 \ldots N-1$, with $N = 2^n$ for n qubits

It is linear and unitary, so it is a valid quantum gate. On a general state it replaces the amplitudes x by their DFT y.

Example: $j = 0$ gives phase 1 for every k, so $\mathrm{QFT}|0\rangle$ is the uniform superposition $(1/\sqrt{N}) \Sigma\, |k\rangle$.

Key idea: every output amplitude has the same size $1/\sqrt{N}$. All the information about j sits in the phases $e^{2\pi i \cdot jk/N}$, which turn faster as j grows.

Phase of each output amplitude, N = 8



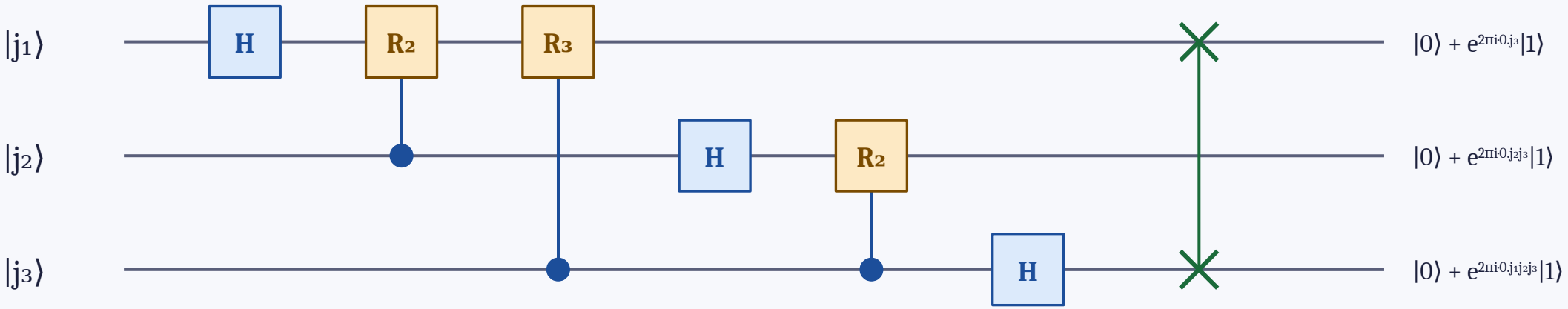
Larger j makes the phase turn faster along k, while every arrow keeps the same length.

2

The QFT circuit: Hadamards and rotations

Each qubit gets a Hadamard, then controlled phase rotations from the qubits below it.

QFT on n = 3 qubits



$R_k = \mathrm{diag}(1, e^{2\pi i/2^{k}})$. Outputs are shown after the swap, each with a factor $1/\sqrt{2}$.

H — Hadamard

Turns qubit j into $(|0\rangle + (-1)^{j}|1\rangle)/\sqrt{2}$, which carries the first binary digit of j as a phase.

R — Controlled-R_k

Adds the next binary digit of j: a rotation by $2\pi/2^{k}$ on the target when the control is $|1\rangle$.

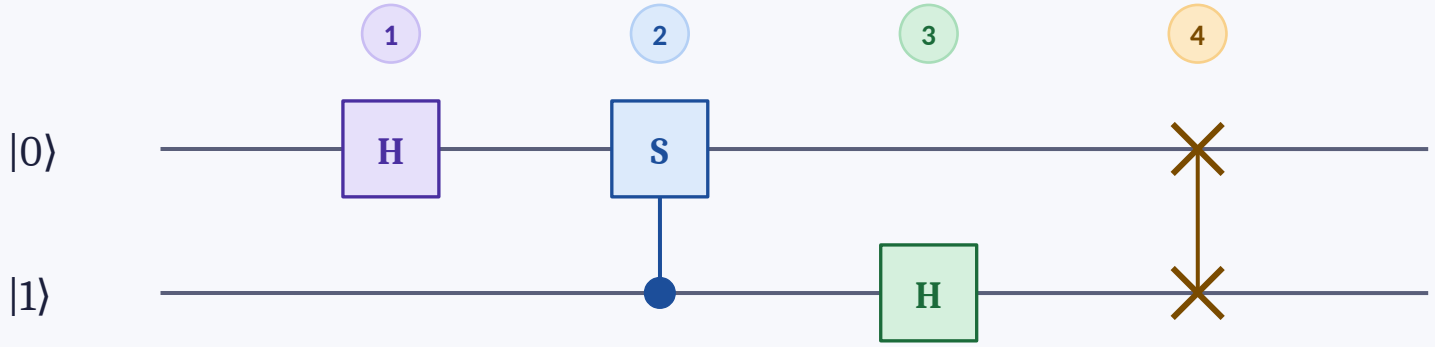
⇄ Swaps

The outputs appear in reverse order, so $\lfloor n/2 \rfloor$ swaps put them back in order.

3

Worked example: a 2-qubit QFT

Follow the input $|01\rangle$ through the circuit one gate at a time.



OUTPUT

$$(|00\rangle + i|01\rangle - |10\rangle - i|11\rangle) / 2$$

1 Hadamard, qubit 1

$$(|0\rangle+|1\rangle)|1\rangle / \sqrt{2}$$

$j_1 = 0$, so qubit 1 gets phase $e^{2\pi i \cdot 0.j_1} = +1$.

2 Controlled-S

$$(|0\rangle+i|1\rangle)|1\rangle / \sqrt{2}$$

Qubit 2 is $|1\rangle$, so $|1\rangle$ on qubit 1 picks up $i = e^{i\pi/2}$.

3 Hadamard, qubit 2

$$(|0\rangle+i|1\rangle)(|0\rangle-|1\rangle) / 2$$

$H|1\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ on qubit 2.

4 SWAP

$$(|0\rangle-|1\rangle)(|0\rangle+i|1\rangle) / 2$$

Exchange the qubits, then expand to get the output above.

3

Check: the 2-qubit QFT matrix

Running every basis state through the circuit gives the columns of the QFT matrix, with ω = i.

QFT matrix for N = 4 (every entry multiplied by ½)

out \ in	|00⟩	|01⟩	|10⟩	|11⟩
|00⟩	1	**1**	1	1
|01⟩	1	**i**	−1	−i
|10⟩	1	**−1**	1	−1
|11⟩	1	**−i**	−1	i

Highlighted column: the input |01⟩ we just traced. Its entries 1, i, −1, −i step by a quarter turn each time.

Entry rule

Row k, column j holds $\omega^{jk} / \sqrt{N}$ with $\omega = e^{2\pi i/4} = i$. For example, row 2, column 1 is $i^2 = -1$.

Matches the circuit

Column |01⟩ is (1, i, −1, −i)/2, exactly the state we reached after the SWAP.

It is unitary

Each column has length 1 and different columns are orthogonal, so the 4 gates built a valid QFT.

4

The inverse QFT

Run the circuit backwards with every rotation reversed, and phases turn back into a basis state.

Definition

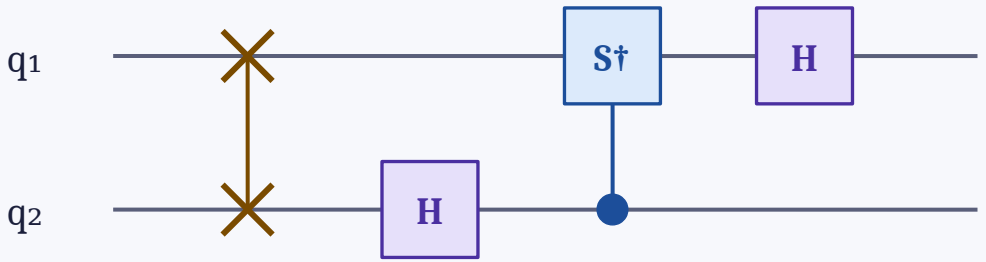
$$\mathrm{QFT}^\dagger|k\rangle = (1/\sqrt{N}) \sum e^{-2\pi i jk/N} |j\rangle$$

the conjugate transpose, so QFT† · QFT = I

1. Reverse the order of the gates, so the swaps come first.
2. Replace each R_k by $R_k\dagger = \mathrm{diag}(1, e^{-2\pi i/2^{k}})$.
3. Leave the Hadamards alone, since H† = H.

Why it matters: QFT† turns phases back into a basis state we can measure.

The 2-qubit QFT†



The forward gates in reverse order. S† = diag(1, −i).

QFT

index j → phases

⇄

QFT†

phases → index j

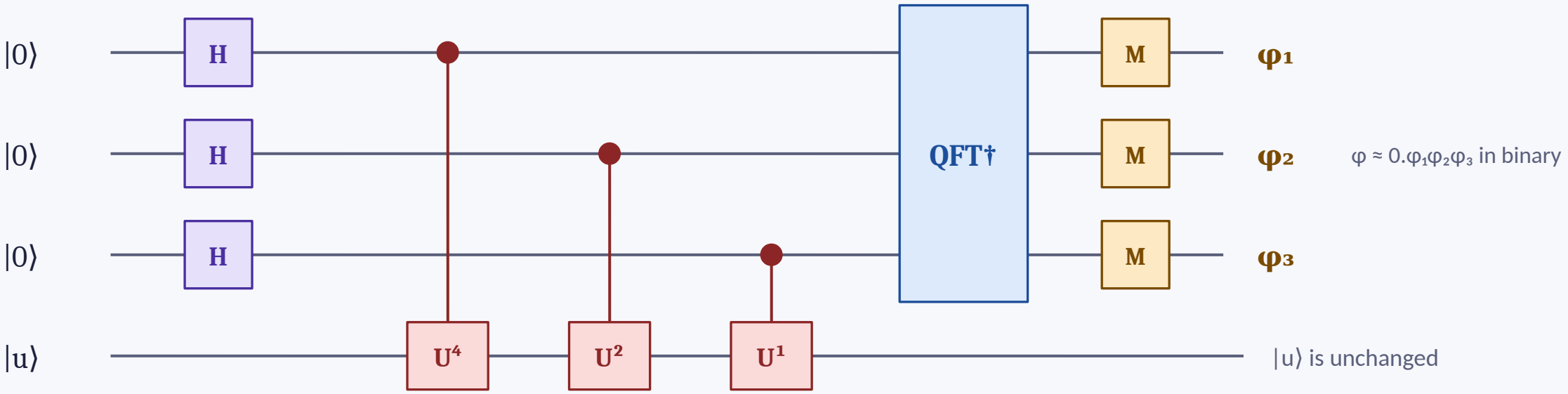
Check: QFT†(QFT|01⟩) = |01⟩.

5

Quantum phase estimation: the circuit

Kick the phase onto the counting qubits, then read it out with the inverse QFT.

Given $U|u\rangle = e^{2\pi i\varphi}|u\rangle$, estimate φ in [0, 1) using t = 3 counting qubits.



1 **Superpose.** Apply H to every counting qubit.

2 **Kick back.** Controlled-$U^{2^{j}}$ writes φ into the phases.

3 **Read out.** Apply QFT†, then measure the counting qubits.

5

Reading the result of phase estimation

After the controlled-U gates φ sits in the phases, and QFT† turns it into bits.

1

After H and the controlled-U gates

$$(1/\sqrt{2^t}) \sum e^{2\pi i \varphi k} |k\rangle|u\rangle$$

The phase φ has been kicked back onto the counting register.

2

If φ = m / 2^t exactly

$$= \mathrm{QFT}\ |m\rangle\ |u\rangle$$

This is a QFT output state, so QFT† undoes it.

3

After QFT† and measurement

$$|m\rangle, \text{so } \varphi = m\ /\ 2^t$$

The measured bits are the binary digits of φ.

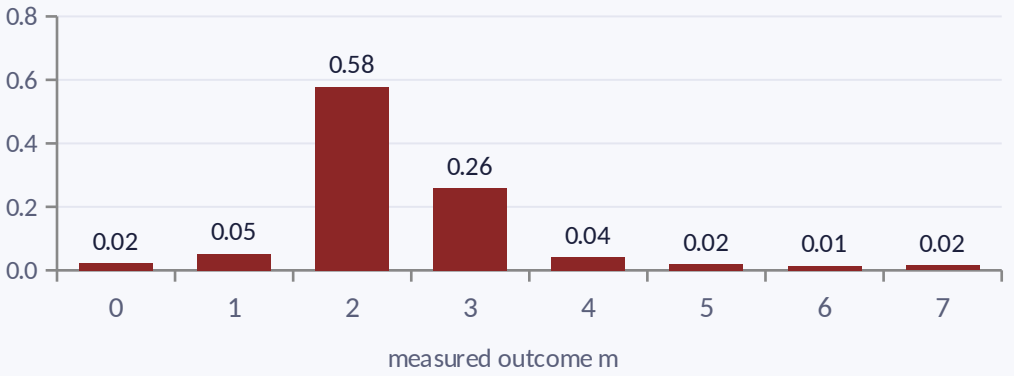
Example: T gate with $|u\rangle = |1\rangle$

$T|1\rangle = e^{i\pi/4}|1\rangle$, so φ = 1/8. With t = 3 the counting register ends in $|001\rangle$.

0	0	1
½	¼	⅛

0.001 in binary = 1/8, so φ = 1/8 exactly.

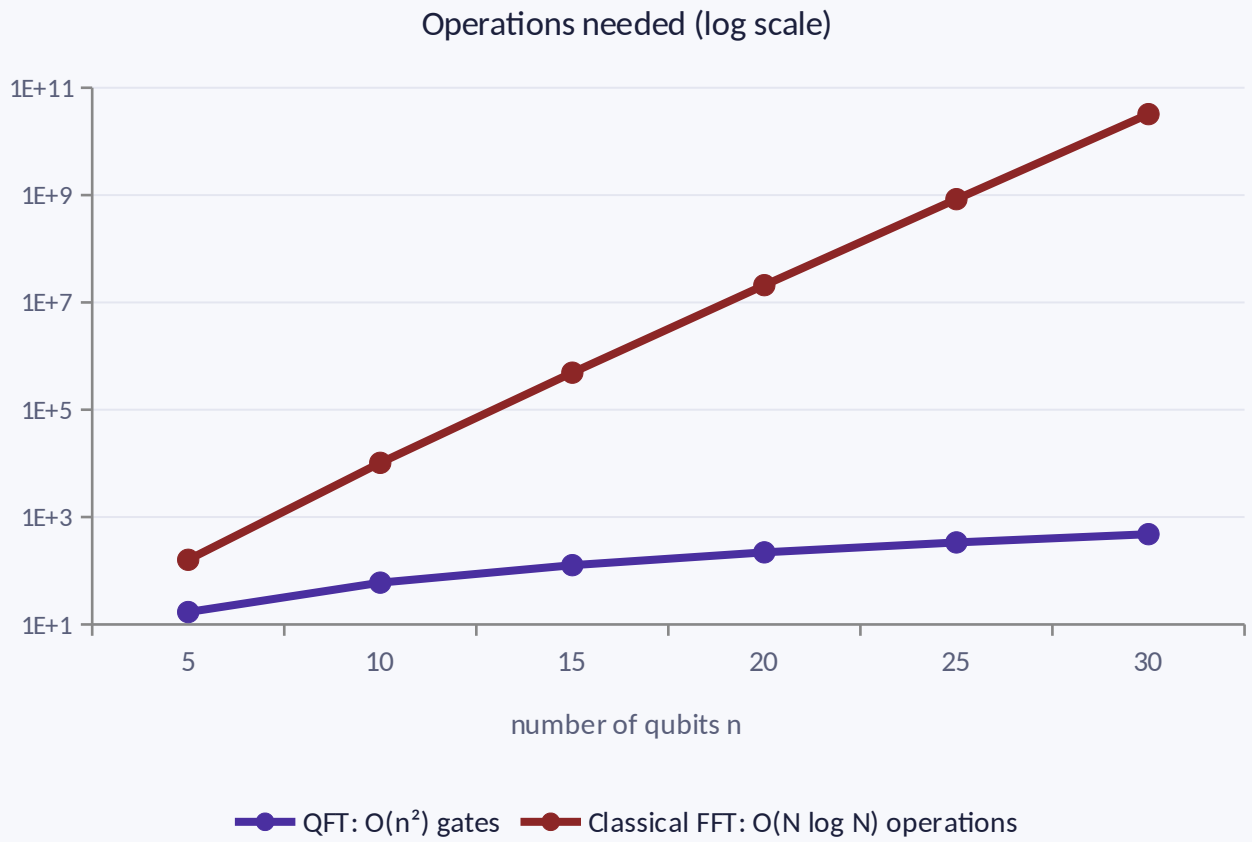
φ = 0.3, t = 3: best outcome m = 2 (φ ≈ 0.25)



6

Complexity: QFT vs classical FFT

For $N = 2^n$ amplitudes, the QFT needs only about n^2 gates.



480

QFT gates at n = 30

3.2×10^{10}

FFT operations at $N = 2^{30}$

Not a free speed-up

The output amplitudes cannot be read out directly. The QFT pays off as a subroutine, as in phase estimation.

Approximate QFT

Dropping the tiniest rotations gives about O(n log n) gates with little error.

ONE-LINE TAKEAWAY

The QFT turns an index into phases using about n^2 gates, and phase estimation runs its inverse to read an eigenvalue's phase as bits.

QFT: index → phases

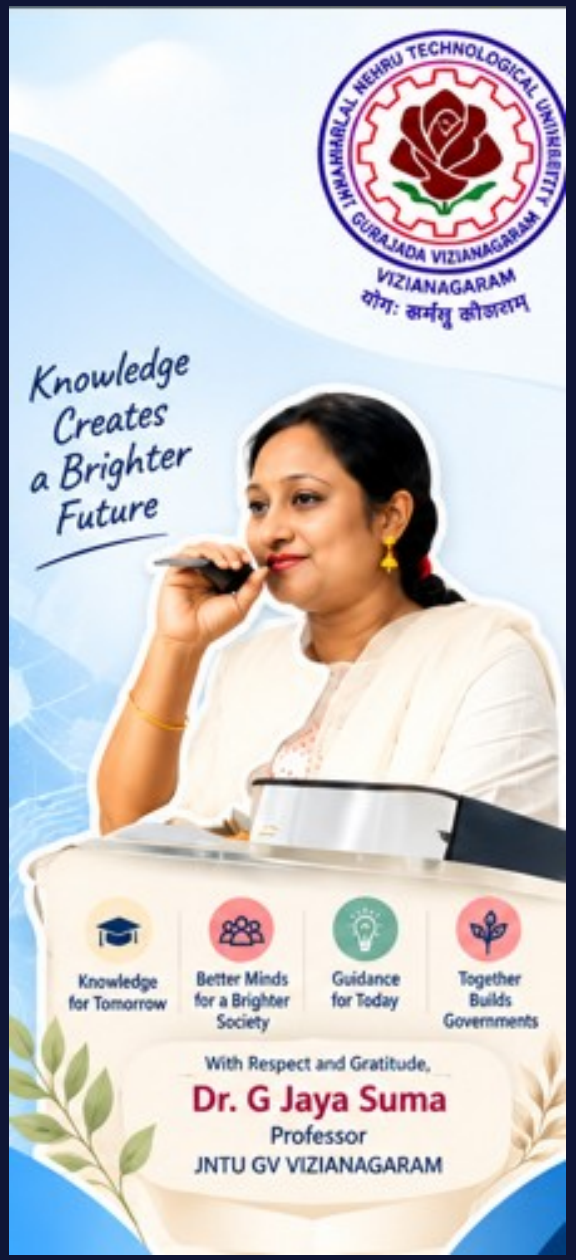
QFT†: phases → index

$O(n^2)$ gates vs $O(N \log N)$

→

Next: Using phase estimation

We put phase estimation to work on a concrete problem and see how it leads to period finding.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Grover's Search Algorithm

Finding one marked item among N in about √N queries — by rotating amplitudes toward the answer.

55 MIN

Oracle | Diffusion | Rotation | Amplify

0

What we'll cover in 55 minutes

Nine segments, from the search problem to the general idea of amplitude amplification.

1 — 5 min

Search & the O(N) baseline

Why N items cost about N queries

2 — 5 min

The phase oracle

A sign flip marks the solution

3 — 7 min

The diffusion operator

Inversion about the mean

4 — 7 min

Geometric picture

Two reflections make a rotation

5 — 5 min

Optimal iterations

Stop near $(\pi/4)\sqrt{N}$ iterations

6 — 4 min

Over-rotation

Why more iterations can hurt

7 — 8 min

Grover in Qiskit

A 2-qubit search, end to end

8 — 7 min

Worked example: N = 16

Classical vs Grover query counts

9 — 7 min

Amplitude amplification

The general framework

1

The unstructured search problem

Find one marked item among N when the only tool is asking “is this the one?”

The problem

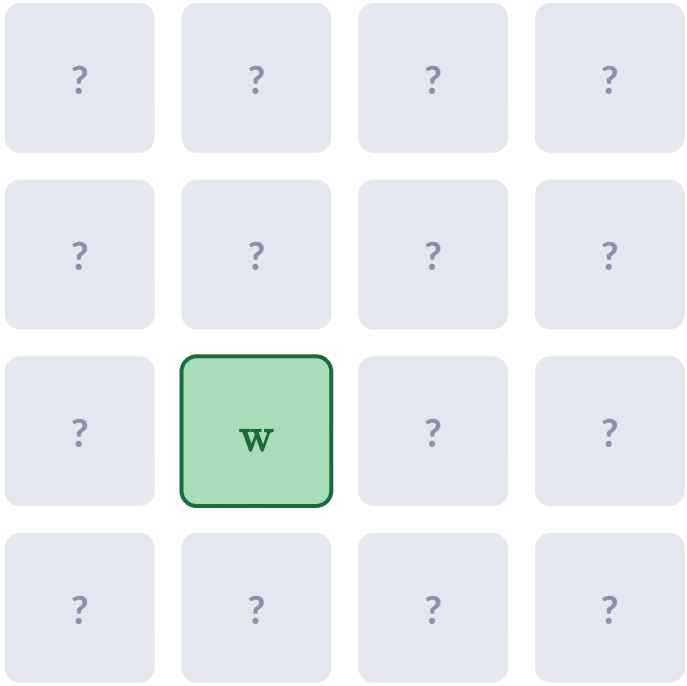
$$f(w) = 1, \quad f(x) = 0 \text{ otherwise}$$

A list of **N = 2^n** items with no order, no index and no structure to exploit. The only access is a black-box check *f* that answers yes for exactly one item, *w*.

Goal: find w with as few calls to f as possible.

Key idea: every call to f is one query, so the cost of a search is counted in queries.

N = 16 items, one marked



We can only open the boxes one at a time.

1

The classical baseline: O(N)

Without structure, the best a classical algorithm can do is check items one by one.

N

Worst case

The marked item is checked last.

N/2

Average case

On average we look through half the list.

q/N

After q queries

Each query rules out one item, nothing more.

Why the classical bound is firm

Items are unrelated, so a “no” for one tells us nothing about the others. Every classical algorithm, even a randomised one, needs on the order of N queries.

The quantum question: can superposition let one query probe all N items at once? Grover’s answer: not quite, but about √N queries are enough.

2

Marking the solution: the phase oracle

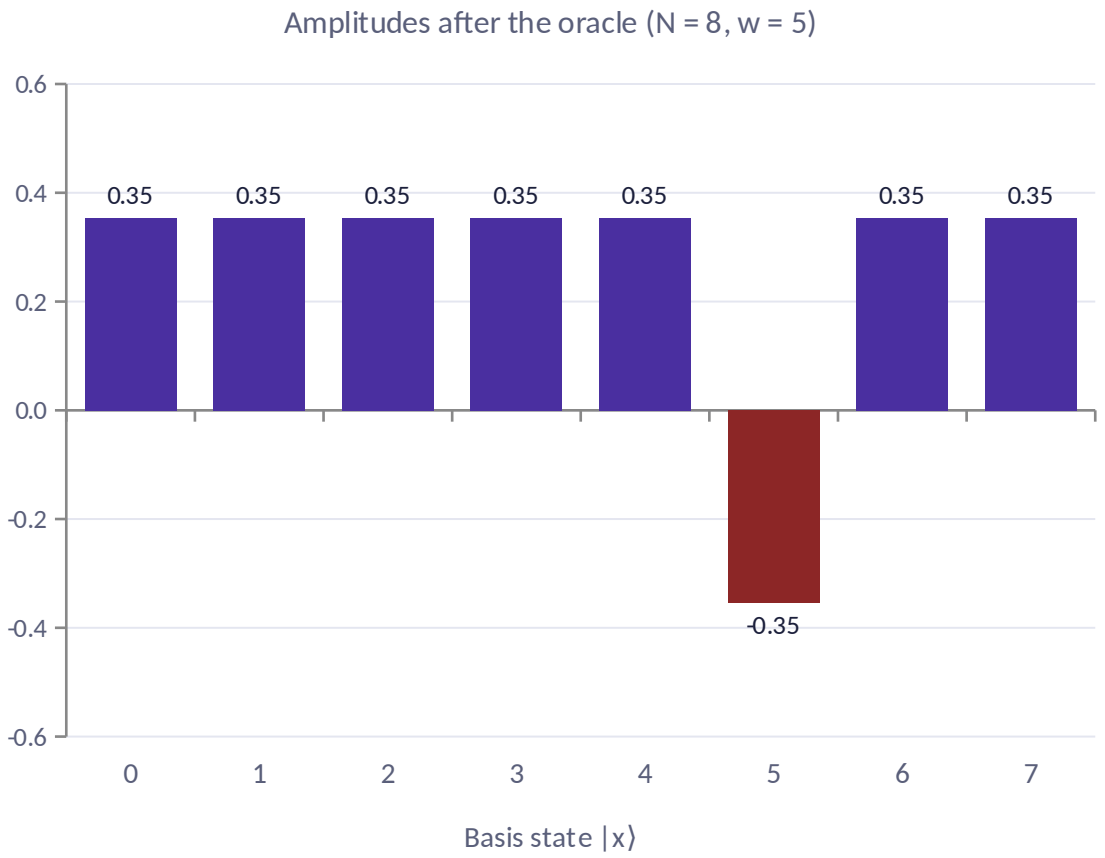
The oracle flips the sign of the solution's amplitude and leaves every other amplitude alone.

The phase oracle

$$U_f|x\rangle = (-1)^{f(x)}|x\rangle$$

If x is not the solution, **f(x) = 0** and nothing changes. If x = w, **f(w) = 1** and the amplitude of $|w\rangle$ gets a minus sign.

Key idea: a sign change cannot be seen by measuring, because probabilities are squares. It matters because the next step turns the sign into a larger amplitude.



2

Building the oracle: phase kickback

An ancilla qubit in $|-\rangle$ turns the usual "flip an output bit" oracle into a sign flip.

1 Bit-flip oracle

$$|x\rangle|y\rangle \to |x\rangle|y \oplus f(x)\rangle$$

The natural reversible way to compute f: XOR f(x) into an output qubit.

2 Ancilla in $|-\rangle$

$$|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$$

Flipping $|-\rangle$ just multiplies it by −1, so the XOR turns into a phase.

3 Sign lands on $|x\rangle$

$$|x\rangle|-\rangle \to (-1)^{f(x)}|x\rangle|-\rangle$$

The ancilla is unchanged and can be ignored; the sign sits on $|x\rangle$.

Cost model: one call to the oracle is one query. Grover's algorithm is judged by how many times it calls the oracle, not by the gates inside it.

N = 4, w = 11: the oracle is a single CZ gate.

3

The diffusion operator

Diffusion reflects every amplitude about the average amplitude: inversion about the mean.

The operator

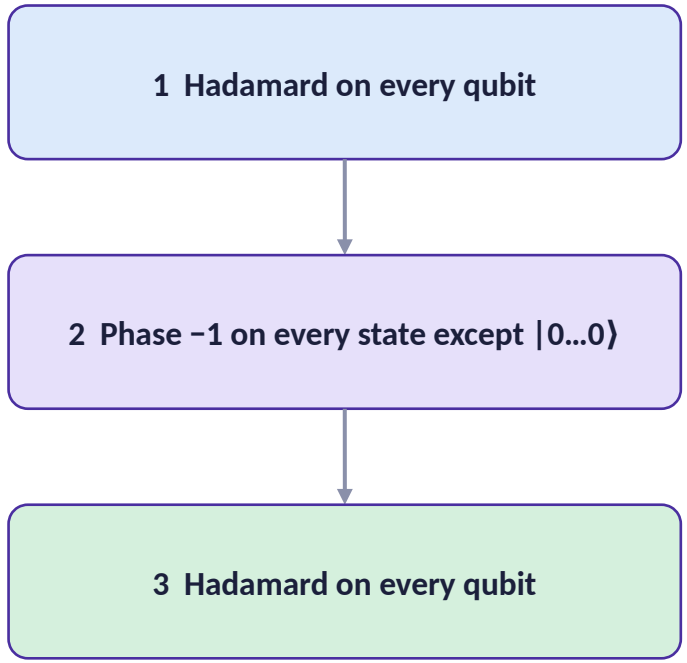
$$D = 2|s\rangle\langle s| - I$$

$|s\rangle$ is the uniform superposition over all N items. D reflects the state about $|s\rangle$; for amplitudes this means **$a \to 2\bar{a} - a$**, where $\bar{a}$ is the mean amplitude.

$$D = H^{\otimes n}\,(2|0\rangle\langle 0| - I)\,H^{\otimes n}$$

Key idea: amplitudes below the mean grow and those above shrink. After the oracle, the marked amplitude is far below the mean, so it grows the most.

The circuit for D

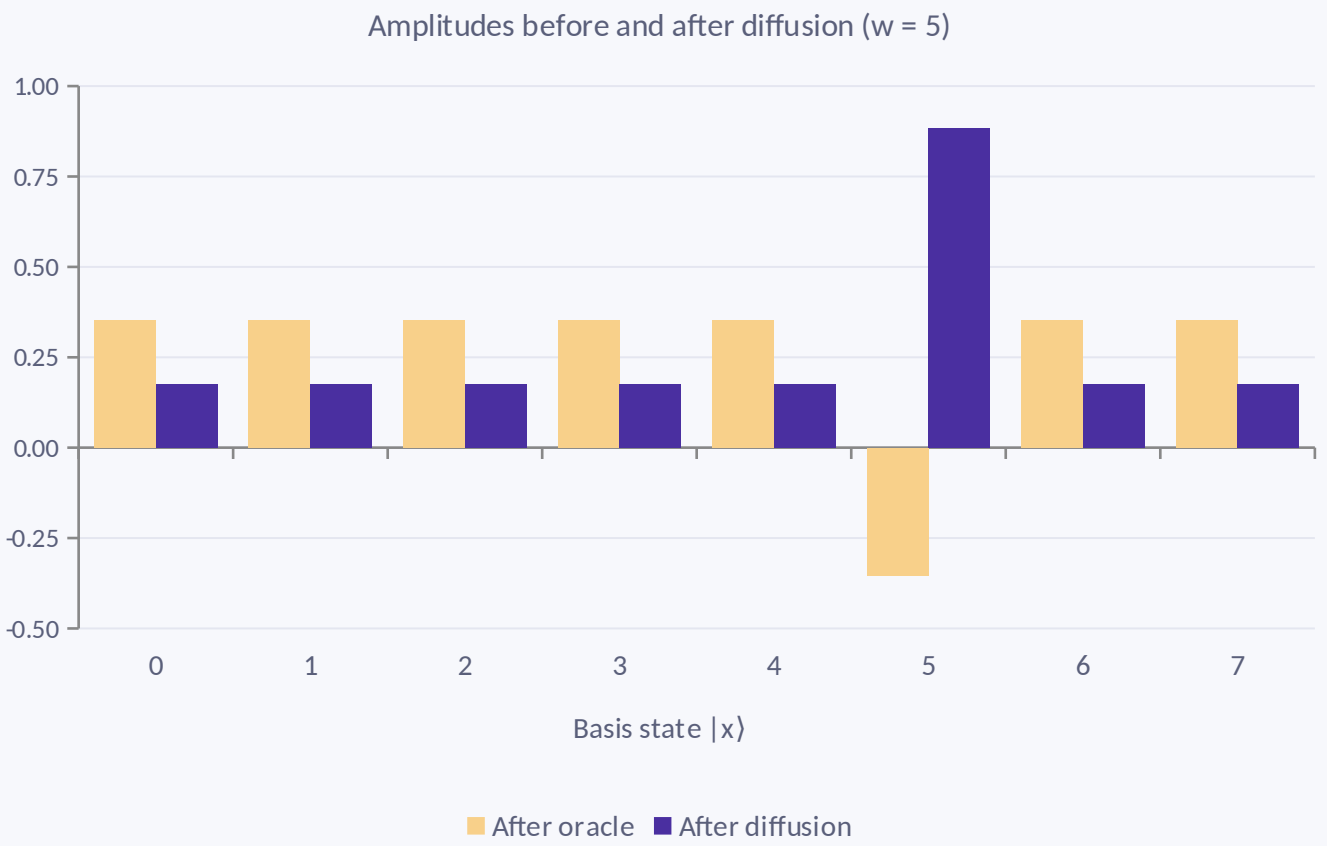


This circuit is exactly $2|s\rangle\langle s| - I$.

3

Inversion about the mean: N = 8

Watch the marked amplitude grow while the others shrink, with the total probability still 1.



1 Find the mean

$\bar{a} = (7 \cdot 0.354 - 0.354) / 8 = 0.265$

2 Other seven states

$2(0.265) - 0.354 = 0.177$, so they shrink

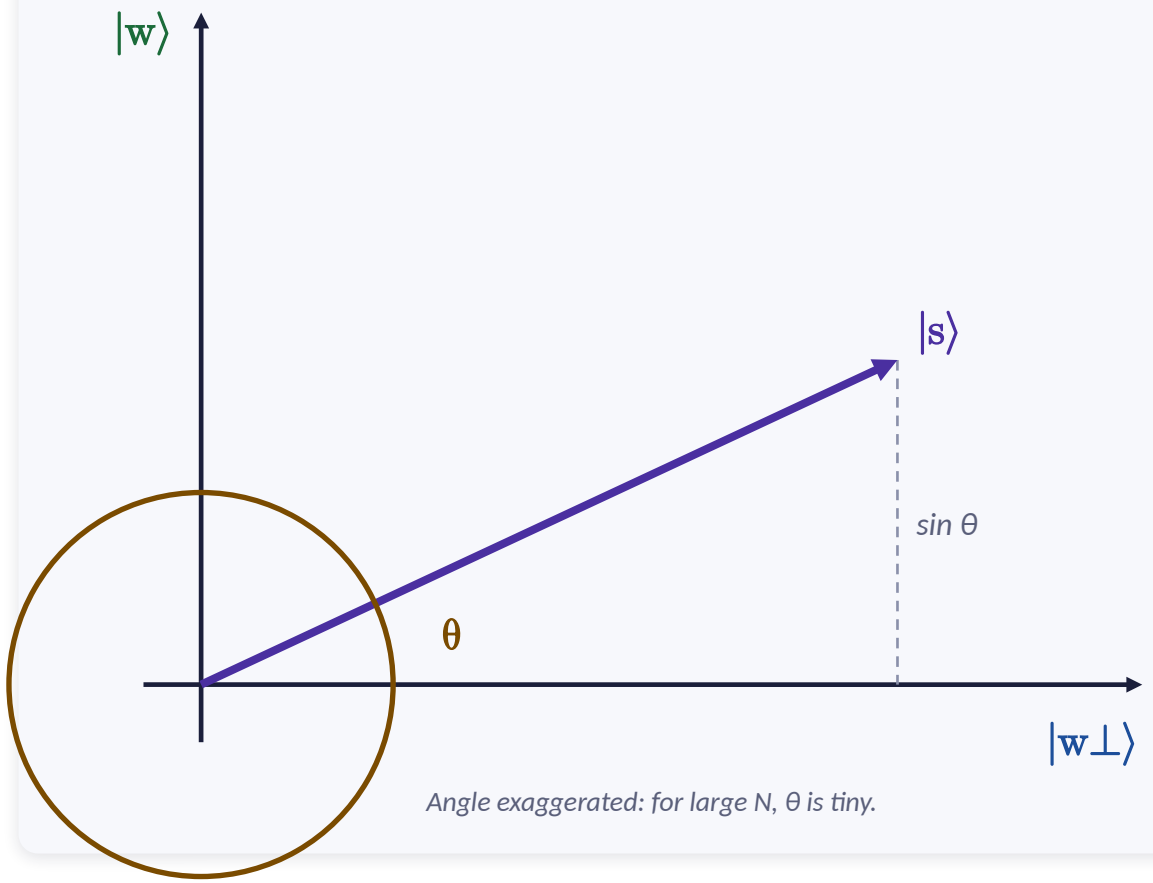
3 Marked state

$2(0.265) + 0.354 = 0.884$, so $P \approx 78\%$

4

Geometric picture: a 2-dimensional subspace

The whole algorithm lives in the plane spanned by the solution $|w\rangle$ and the uniform "rest" $|w\perp\rangle$.



Angle exaggerated: for large N, θ is tiny.

$|w\rangle$

The solution

The one state we want to find.

$|w\perp\rangle$

Everything else

The equal superposition of the other N − 1 states.

$|s\rangle$

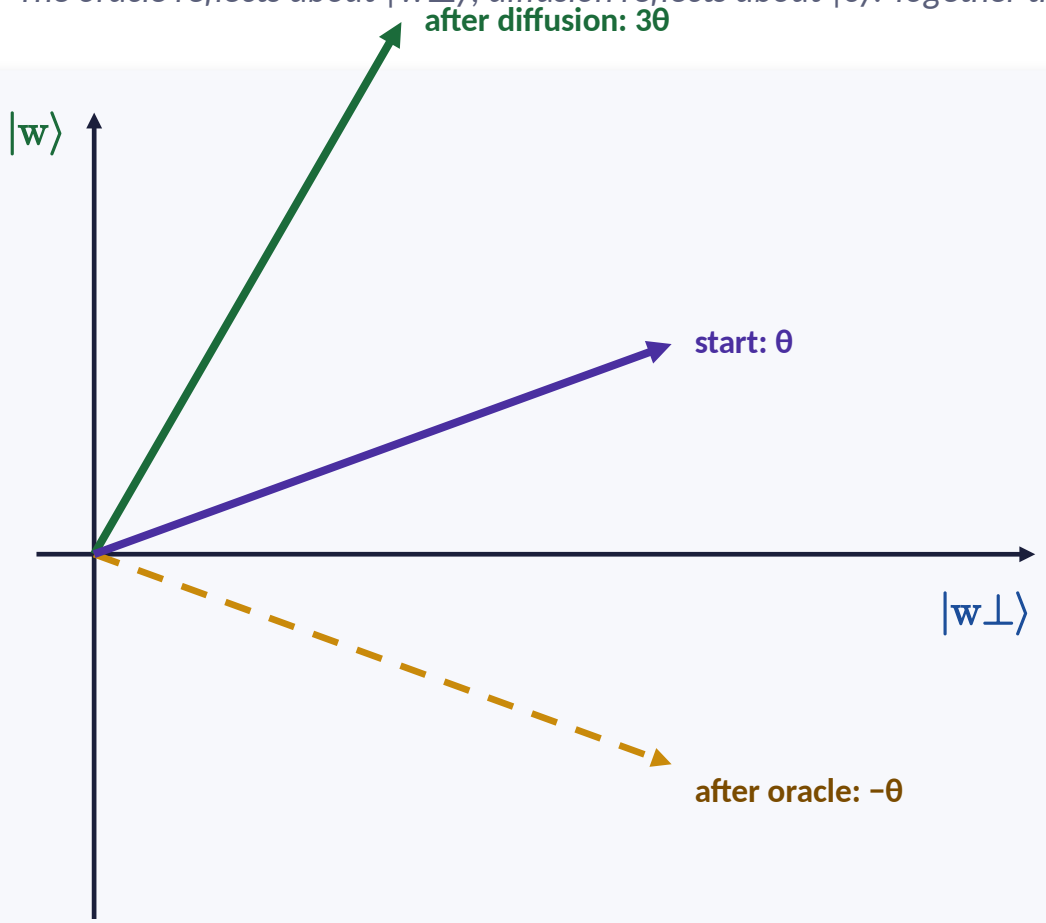
The start state

Lies at angle θ above $|w\perp\rangle$, with $\sin\theta = 1/\sqrt{N}$.

4

Two reflections make a rotation by 2θ

The oracle reflects about |w⊥⟩; diffusion reflects about |s⟩. Together they rotate toward |w⟩.



Angles exaggerated for clarity.

1 Oracle

Reflects about |w⊥⟩: angle θ becomes −θ.

2 Diffusion

Reflects about |s⟩: angle −θ becomes 3θ.

After k iterations

$$|\psi_k\rangle = \sin((2k+1)\theta)|w\rangle + \cos((2k+1)\theta)|w^\perp\rangle$$

5

Choosing the optimal number of iterations

Stop when the state is as close to $|w\rangle$ as possible: $(2k + 1)\theta \approx \pi/2$.

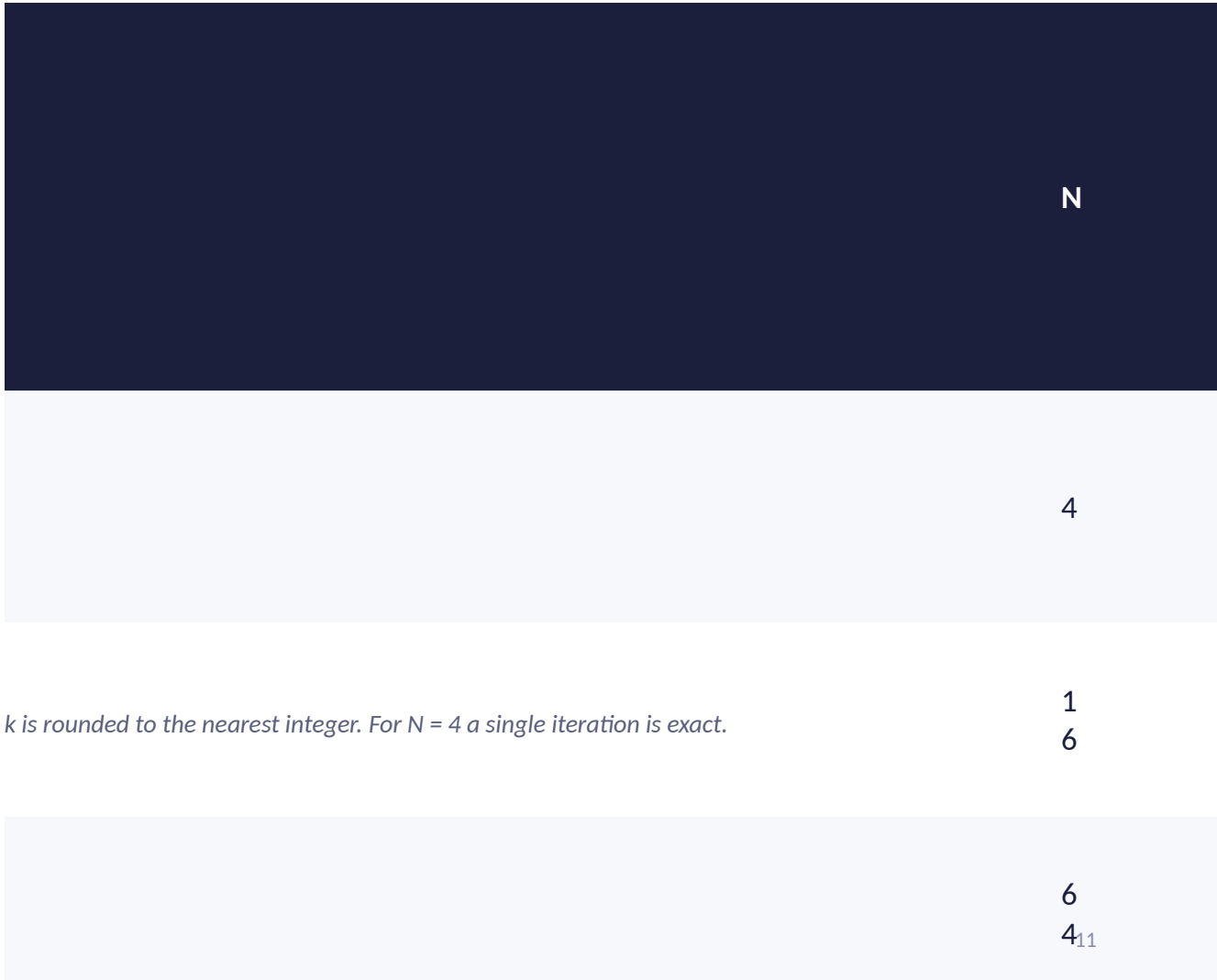
Where to stop

$$k \approx (\pi/4)\sqrt{N}$$

After k iterations the angle is $(2k + 1)\theta$, so **$P(k) = \sin^2((2k + 1)\theta)$**.
Setting $(2k + 1)\theta = \pi/2$ and using $\theta \approx 1/\sqrt{N}$ gives the formula above.

Quadratic speedup: about $\sqrt{N}$ queries instead of about N.

Optimal k for different N

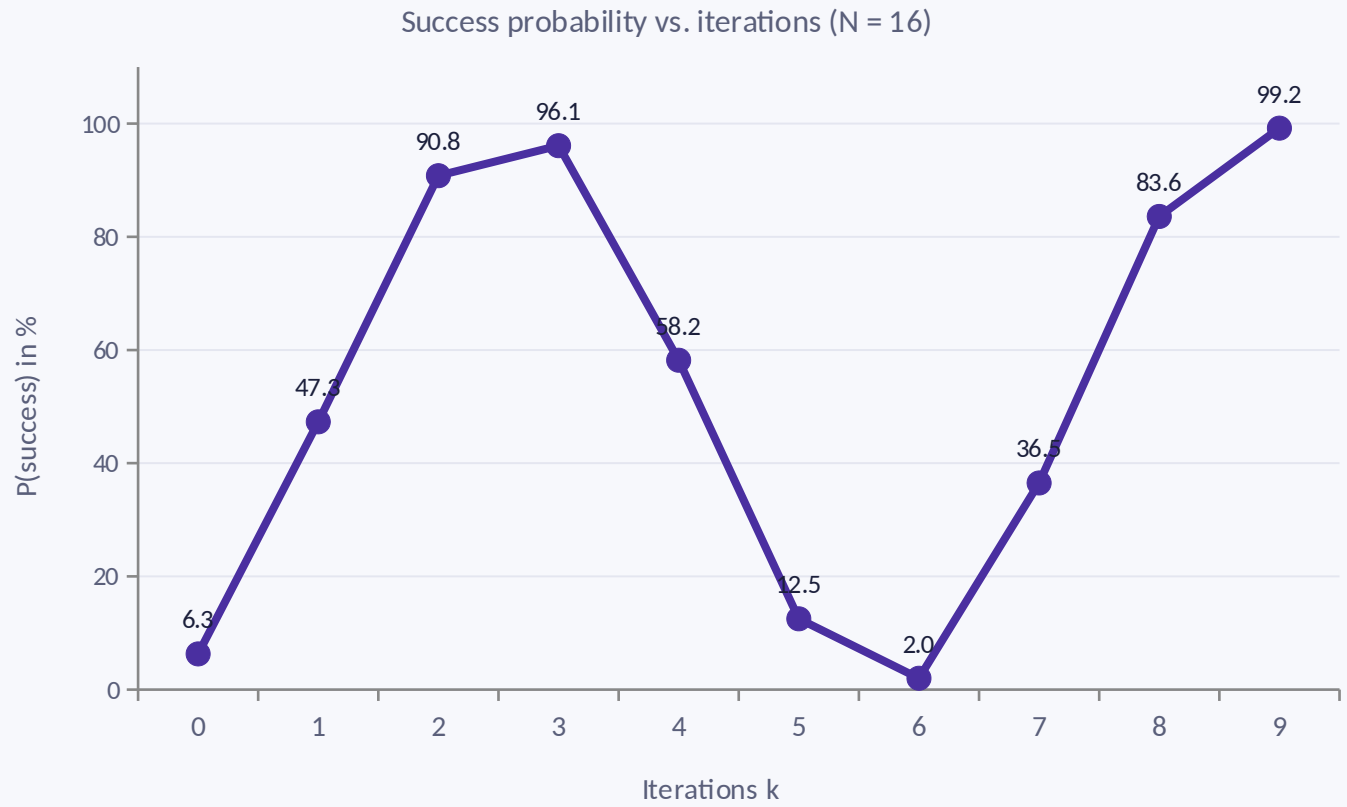


k is rounded to the nearest integer. For N = 4 a single iteration is exact.

6

What happens if you over-rotate

Past the peak the state rotates beyond |w⟩ and the success probability falls again.



k = 3: 96.1%

The optimum: the state sits about 11° past 90°.

k = 4: 58.2%

One step too many: the rotation has overshot |w⟩.

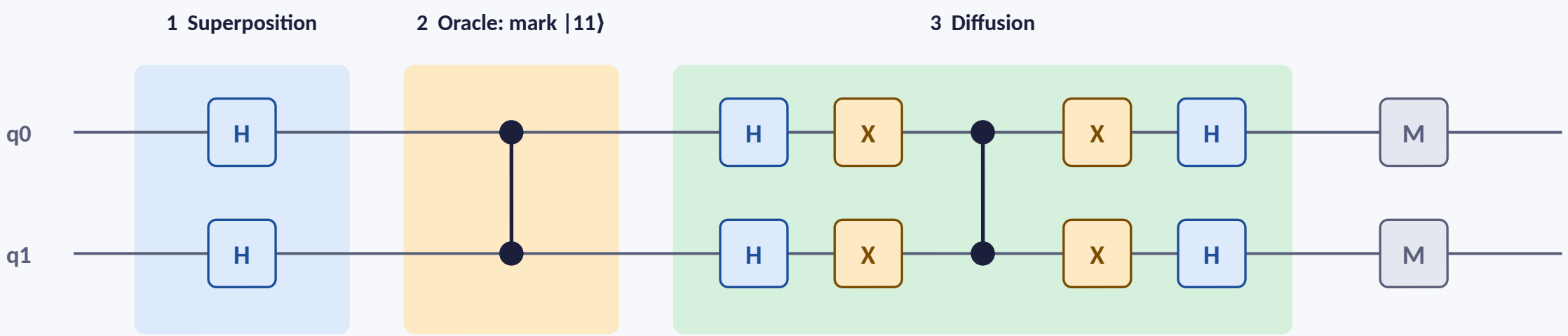
k = 6: 2.0%

Almost back at the start, so the answer is nearly gone.

7

Grover on a 2-qubit search space

N = 4 with the marked item |11⟩: a single iteration is exactly enough.



n = 2, N = 4

Two qubits, four items, one marked.

θ = 30°

$\sin\theta = 1/\sqrt{4} = 1/2$, so $\theta = 30°$.

k = 1

$\sin^2(3\theta) = \sin^2(90°) = 100\%$.

7

Implementing it in Qiskit

Four short blocks: superposition, oracle, diffusion, then read the probabilities.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

qc = QuantumCircuit(2)
qc.h([0, 1])                  # 1. superposition

qc.cz(0, 1)                   # 2. oracle: flip |11>

qc.h([0, 1])                  # 3. diffusion
qc.x([0, 1])
qc.cz(0, 1)
qc.x([0, 1])
qc.h([0, 1])

print(Statevector(qc).probabilities_dict())
```

1 Superposition

H on both qubits gives amplitude ½ on every state.

2 Oracle

CZ flips the sign of $|11\rangle$ only, the marked state.

3 Diffusion

H, X, CZ, X, H inverts every amplitude about the mean.

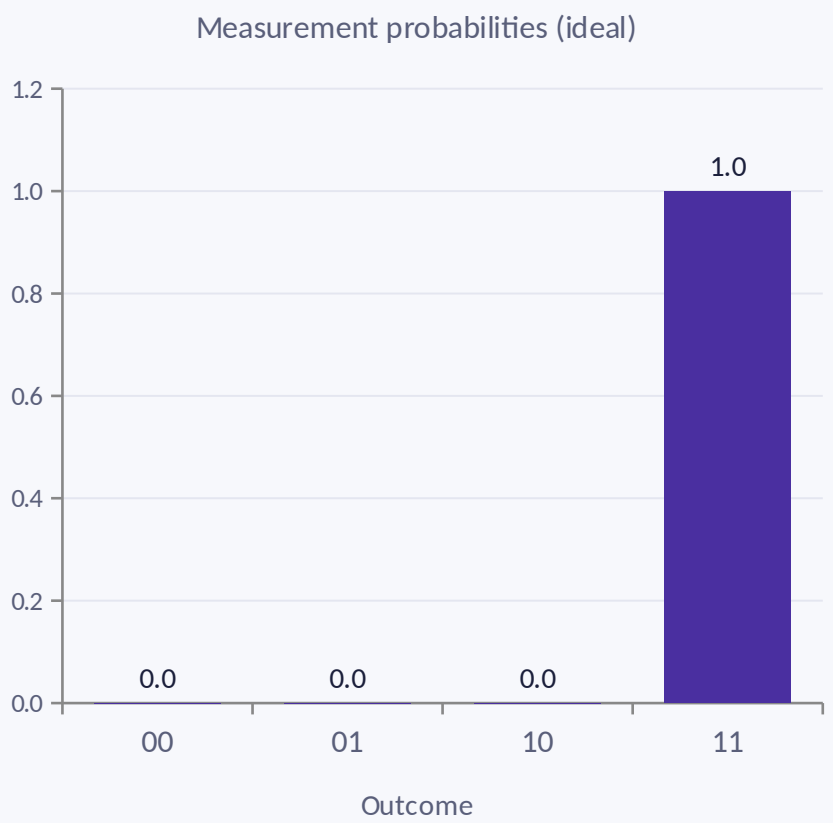
To sample instead, add qc.measure_all() and run on a simulator.

7

Result: the marked state is found with certainty

Tracing the amplitudes shows why one iteration is exactly enough for N = 4.

	Stage
After H	
After oracle	
After diffusion	



Why it works

After the oracle the mean amplitude is ¼. Diffusion maps **½ → 2(¼) − ½ = 0** and **−½ → 2(¼) + ½ = 1**. One oracle call is enough.

8

Worked example: N = 16, step by step

Four qubits and one marked item: θ = arcsin(1/4) ≈ 14.48°, so we use k = 3 iterations.

k

16

items searched

3

oracle calls (queries)

96.1%

success probability

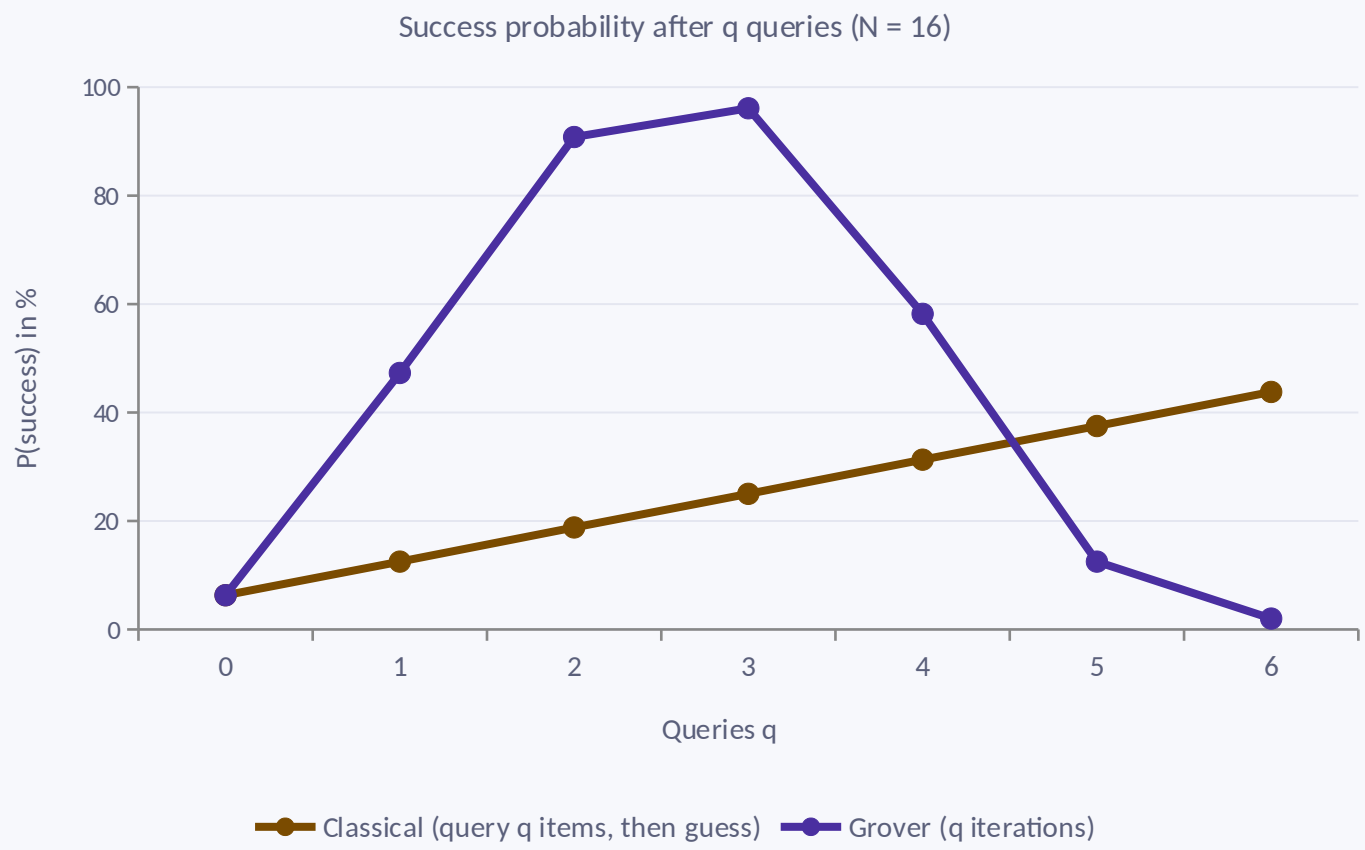
Why k = 3, not 2? 7θ = 101.3° is 11.3° from 90°, while 5θ = 72.4° is 17.6° away. The formula (π/4)√16 ≈ 3.14 agrees.

1

8

Classical vs Grover: query counts for N = 16

After 3 queries, a classical search succeeds 25% of the time; Grover succeeds 96% of the time.



15 — **Classical, worst case**

queries to be certain of finding w.

≈ 8.5 — **Classical, on average**

queries to find w with certainty.

3 — **Grover**

queries for 96.1% success.

9

Amplitude amplification: the general framework

Grover's trick works for any procedure that succeeds with a small probability.

The general recipe

$$A|0\rangle = \sin\theta\ |\text{good}\rangle + \cos\theta\ |\text{bad}\rangle$$

Start from any circuit A that outputs a good answer with probability **$p = \sin^2\theta$**. Repeat two reflections: flip the sign of the good states, then reflect about $A|0\rangle$. Each round rotates by 2θ.

Key idea: about $(\pi/4)/\sqrt{p}$ rounds make the answer near-certain.

Grover as a special case



9

What amplitude amplification gives us

A quadratic speedup for many tasks, and a hard limit on how much more is possible.

Grover is optimal: no quantum algorithm can search an unstructured list with fewer than about $\sqrt{N}$ queries.

A

Where it is used

Speeds up any procedure that succeeds with probability p: 1/p repeats become about $1/\sqrt{p}$. Examples: search problems, minimum finding, collision finding.

M

Many solutions

With M solutions, $\sin\theta = \sqrt{(M/N)}$ and $k \approx (\pi/4)\sqrt{(N/M)}$. If M is unknown, quantum counting or randomised stopping finds it.

!

Limits to remember

The speedup is quadratic, not exponential. The oracle must be cheap, and over-rotating loses the answer.

ONE-LINE TAKEAWAY

Grover’s algorithm alternates an oracle sign flip with inversion about the mean, rotating the state by 2θ each round, so about $(\pi/4)\sqrt{N}$ queries find the marked item.

Oracle: flip the sign

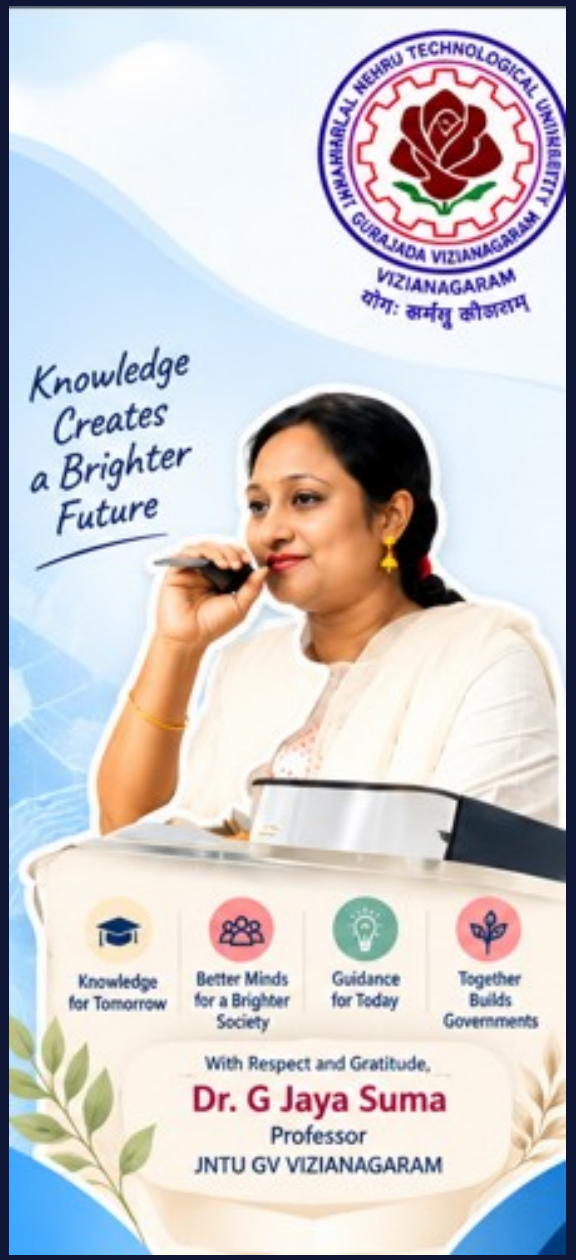
Diffusion: invert about the mean

Stop near $(\pi/4)\sqrt{N}$

→

Next topic in this unit

We carry the amplitude-amplification idea forward and look at where else quantum speedups come from.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Shor's Factoring Algorithm

Turning factoring into period-finding — an exponential speedup that undermines RSA encryption.

50 MIN | Shor's Algorithm | Period-finding | QFT | Exponential speedup

1 What we'll cover in 50 minutes

Seven segments, from recapping the Fourier transform to why factoring breaks RSA.

1 — 5 min

Recap: QFT & phase estimation

Reading out phase, and why period-finding beats amplitude search here.

2 — 5 min

Factoring, RSA & period-finding

Turning “factor N” into “find the period of a^x mod N.”

3 — 10 min

Shor's circuit: exponentiation + QFT

Compute a^x mod N in superposition, then apply the inverse QFT.

4 — 10 min

Frequency picture: peaks at Q/r

Measurement lands near multiples of Q/r — a comb of narrow peaks.

5 — 5 min

Continued fractions: recovering r

Turning one noisy measured value back into the exact period.

6 — 10 min

Worked example: factoring 15

Trace a = 7 through the pipeline and recover the factors 3 and 5.

7 — 5 min

Why it matters: impact & limits

An exponential speedup that breaks RSA — if hardware ever scales.

2 Recap: the Fourier transform and phase estimation

Two tools from earlier episodes make Shor's algorithm possible.

In simple words

$$U|u\rangle = e^{2\pi i\varphi}|u\rangle \quad \rightarrow \quad \text{estimate } \varphi$$

Shor reuses two ideas from earlier topics: quantum phase estimation, which reads out an eigenvalue's phase, and the quantum Fourier transform, which turns a periodic amplitude pattern into a handful of sharp peaks.

Phase estimation writes φ, in binary, into a register using the inverse QFT.

A periodic sequence has phase φ = k/r — the QFT concentrates it into peaks near multiples of 1/r.

U

Phase estimation

Estimates the phase φ in $U|u\rangle = e^{2\pi i\varphi}|u\rangle$ to n bits of precision.

F

Quantum Fourier Transform

Maps a state periodic with period r to amplitude spikes spaced Q/r apart.

r

Why period, not amplitude

Shor cares about the period r of $a^x \bmod N$ — not a single marked item's probability.

3 Factoring, RSA, and the period-finding problem

Turn "factor N" into "find the period of a^x mod N" — then let a quantum computer find it.

The problem

$$N = p \cdot q \;\rightarrow\; \text{find a nontrivial factor of } N$$

RSA's security assumes factoring is classically hard: the best known classical method (the general number field sieve) runs in sub-exponential time.

Classical: sub-exponential time. **Quantum:** $O((\log N)^3)$.

Note: the reduction is exact — a fast period-finder gives a fast factorer via one gcd computation.

A periodic sequence: a^x mod N



Example: a = 7, N = 15 — the sequence repeats every 4 steps, but nothing reveals that fact until you look for the period.

4 Shor's circuit: modular exponentiation and inverse QFT

One control register, one work register, and a single measurement.

One Shor run

$$|\psi\rangle = \mathrm{QFT}^\dagger \cdot U_a \cdot (H^{\otimes n} \otimes I)|0\rangle^{n}|1\rangle$$

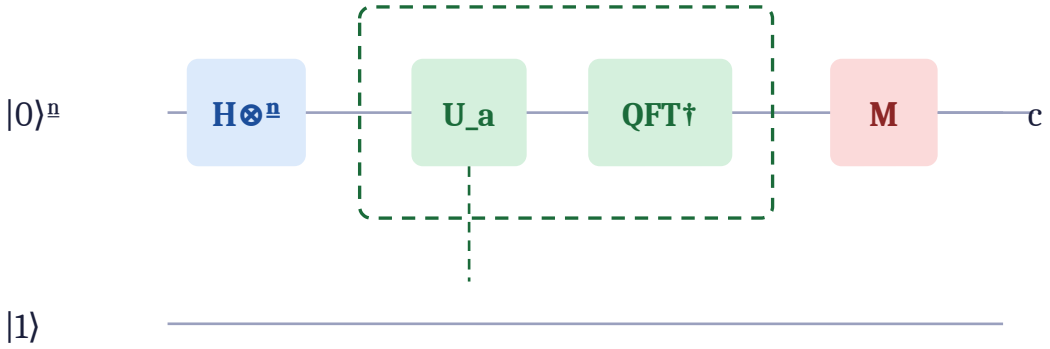
1. Superposition — H on every qubit of the control register.

2. Modular exponentiation U_a — compute a^x mod N into the work register, entangling it with x.

3. Inverse QFT on the control register — concentrates amplitude near multiples of Q/r, then measure.

Note: unlike Grover, there's no repeated rotation — one run either succeeds or is repeated from scratch with a new a.

The Shor circuit



H builds the superposition; U_a entangles x with a^x mod N; the inverse QFT turns period into frequency.

5 The frequency picture: peaks at multiples of Q/r

Measuring the control register lands near a multiple of Q divided by the period.

The frequency-domain picture

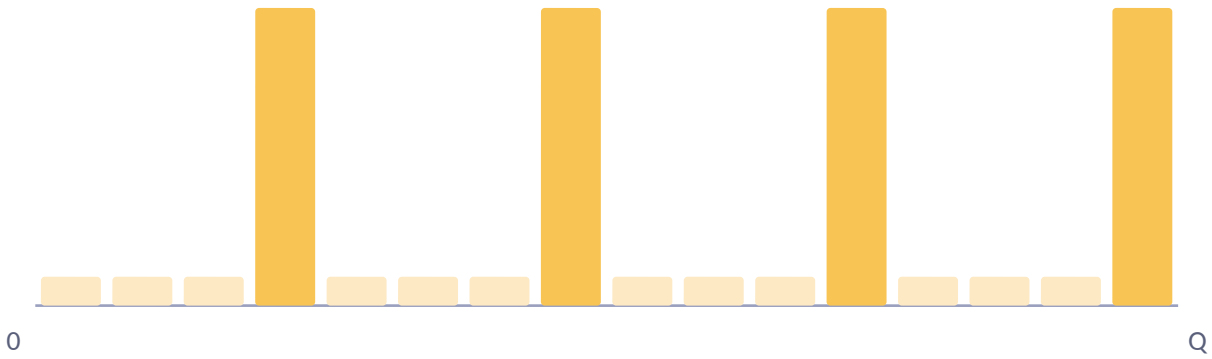
$$\text{measured value } c \approx k \cdot Q/r, \quad Q = 2^n$$

After the inverse QFT, probability piles up in a comb of peaks spaced Q/r apart — one for each integer k. A single measurement returns one of these peaks.

A larger period r gives more, closely-spaced peaks; a smaller r gives fewer, wider-spaced peaks.

Careful: the returned value only sits close to k·Q/r, not exactly on it — continued fractions (next segment) recovers r from it.

Amplitude after the inverse QFT



Peaks sit at multiples of Q/r (here r = 4); everywhere else amplitude nearly cancels.

6 Continued fractions: recovering the period r

Classical post-processing turns one noisy measurement into the exact period.

The problem

$$c / Q \;\rightarrow\; \text{continued fraction} \;\rightarrow\; k / r$$

Expand c/Q as a continued fraction and read off its convergents k/r — fractions in lowest terms. The denominator of the best convergent with r < N is very likely the true period.

This step runs on an ordinary classical computer, once c has been measured.

Note: if r is odd, or a^(r/2) ≡ −1 (mod N), the run is a dud — just try again with a fresh a.

Example: Q = 16, measured c = 12

c / Q	**12 / 16**
reduce	**3 / 4**
convergent k / r	**k = 3, r = 4**

r = 4 matches the true period of a = 7 mod 15, seen back in the periodic sequence.

7 Worked example: factoring 15

Trace a = 7 through the whole pipeline and recover the factors 3 and 5.

Example: N = 15, a = 7, measured period r = 4

a^(r/2) mod N = 7^2 mod 15 = 49 mod 15 = 4

gcd(4 − 1, 15) = gcd(3, 15) = 3 and gcd(4 + 1, 15) = gcd(5, 15) = 5

Two gcd calls, run on a classical computer, split 15 into its prime factors — no further quantum steps needed.

The sequence 7^x mod 15

x=0	x=1	x=2	x=3
1	7	4	13

The period is 4

Four steps before the sequence repeats — an even period, as required.

One pair of gcd calls

Two gcd computations turn the period directly into both nontrivial factors.

Not guaranteed on one try

Roughly half of random choices of a succeed immediately; a dud run just restarts.

8 Why it matters: impact and limits

A guaranteed exponential speedup — that breaks RSA, if the hardware ever scales.

The headline result

Shor's algorithm factors an n-digit number in $O((\log N)^3)$ time — exponentially faster than the best known classical method, and it breaks RSA and similar cryptosystems that rely on factoring's hardness.

$O((\log N)^3)$ — exponential speedup over classical factoring

This isn't only theoretical: an adversary with a large enough fault-tolerant quantum computer could break today's RSA-2048 keys.

Note: today's quantum hardware is far from the many clean logical qubits Shor's algorithm needs — but the mathematics of the attack is already settled.

e^x

Exponential, not quadratic

A bigger gap than Grover's — factoring moves from sub-exponential to polynomial time.

QEC

Needs fault tolerance

Long, deep circuits require error-corrected logical qubits, not today's noisy devices.

PQC

Motivates post-quantum crypto

Standards bodies are already moving to lattice- and hash-based schemes immune to Shor's attack.

ONE-LINE TAKEAWAY

Modular exponentiation plus an inverse Fourier transform turns factoring into period-finding, solvable in about $(\log N)^3$ steps — an exponential speedup that undermines RSA.

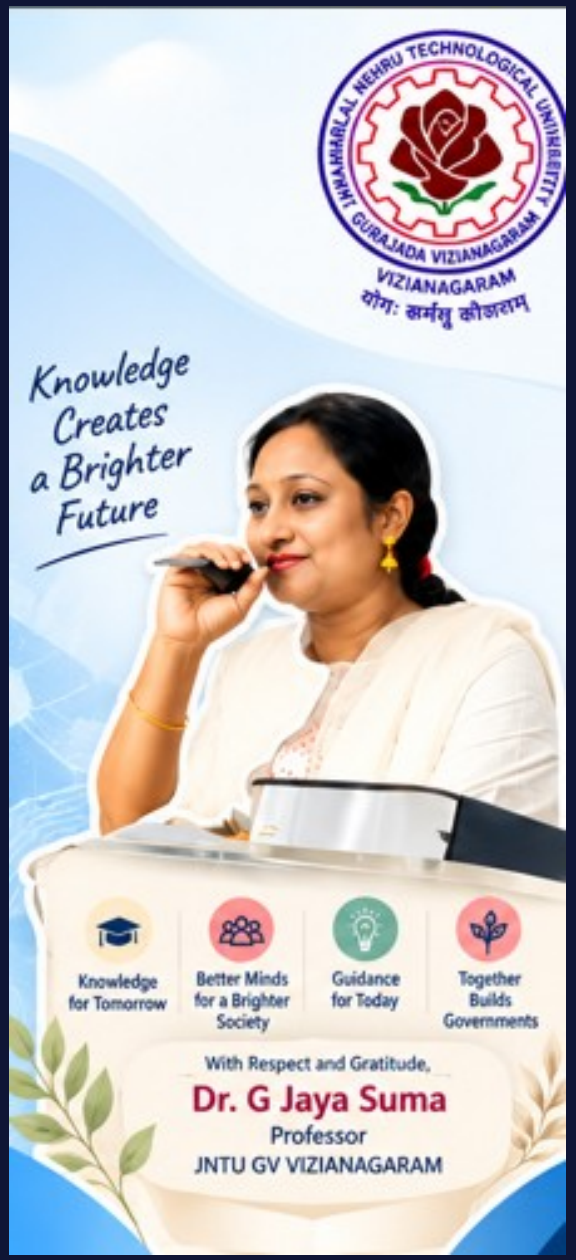
O((log N)³) time

Exponential speedup

Modular exp + QFT†

Next: Quantum Error Correction

We've seen the mathematics that breaks RSA — next we turn to the noisy hardware problem: keeping fragile qubits reliable long enough to run it.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Speedup Analysis & Limits of Quantum Advantage

When does a quantum algorithm actually win — and by how much does it really matter?

35 MIN | Complexity classes | Query lower bounds | NISQ limits

1 What we'll cover in 35 minutes

Seven short segments, from recapping past speedups to a framework for spotting real ones.

1 — 5 min

Recap: the speedups we've seen

From Grover's √N to Shor's exponential leap — a quick tour.

2 — 5 min

Classifying speedups: complexity classes

Where quantum algorithms sit relative to P, BPP, and BQP.

3 — 5 min

Why √N is optimal: a quantum lower bound

A proof that no algorithm can beat Grover's quadratic speedup.

4 — 5 min

Where quantum wins big

Structured problems — factoring, simulation — with exponential gaps.

5 — 5 min

Where quantum doesn't help

Most problems have no known quantum speedup at all.

6 — 5 min

The catch: overhead & NISQ limits

Why today's noisy hardware can't yet cash in these speedups.

7 — 5 min

Why it matters: real quantum advantage

A framework for asking whether a claimed speedup is real.

2 Recap: the speedups we've seen so far

Two very different flavors of quantum speedup, from earlier in this series.

In simple words

$$\text{Grover: } O(\sqrt{N}) \qquad \text{Shor: } O((\log N)^3)$$

This series has shown two different kinds of quantum speedup: Grover's quadratic speedup for unstructured search, and Shor's exponential speedup for factoring. Both beat the best classical method — but by very different margins.

A quadratic speedup helps, but classical computers stay competitive for a long time as N grows.

An exponential speedup eventually makes classical computation hopeless, no matter how much hardware you throw at it.

√

Quadratic (Grover)

N → √N queries — a real but modest win; the gap grows slowly with N.

e

Exponential (Shor)

Sub-exponential → polynomial time — a gap that explodes as N grows.

?

Not all speedups are equal

The size of the gap — and how it scales — decides whether a speedup ever matters in practice.

3 Classifying speedups: where quantum sits

Complexity classes group problems by how much time they need to solve.

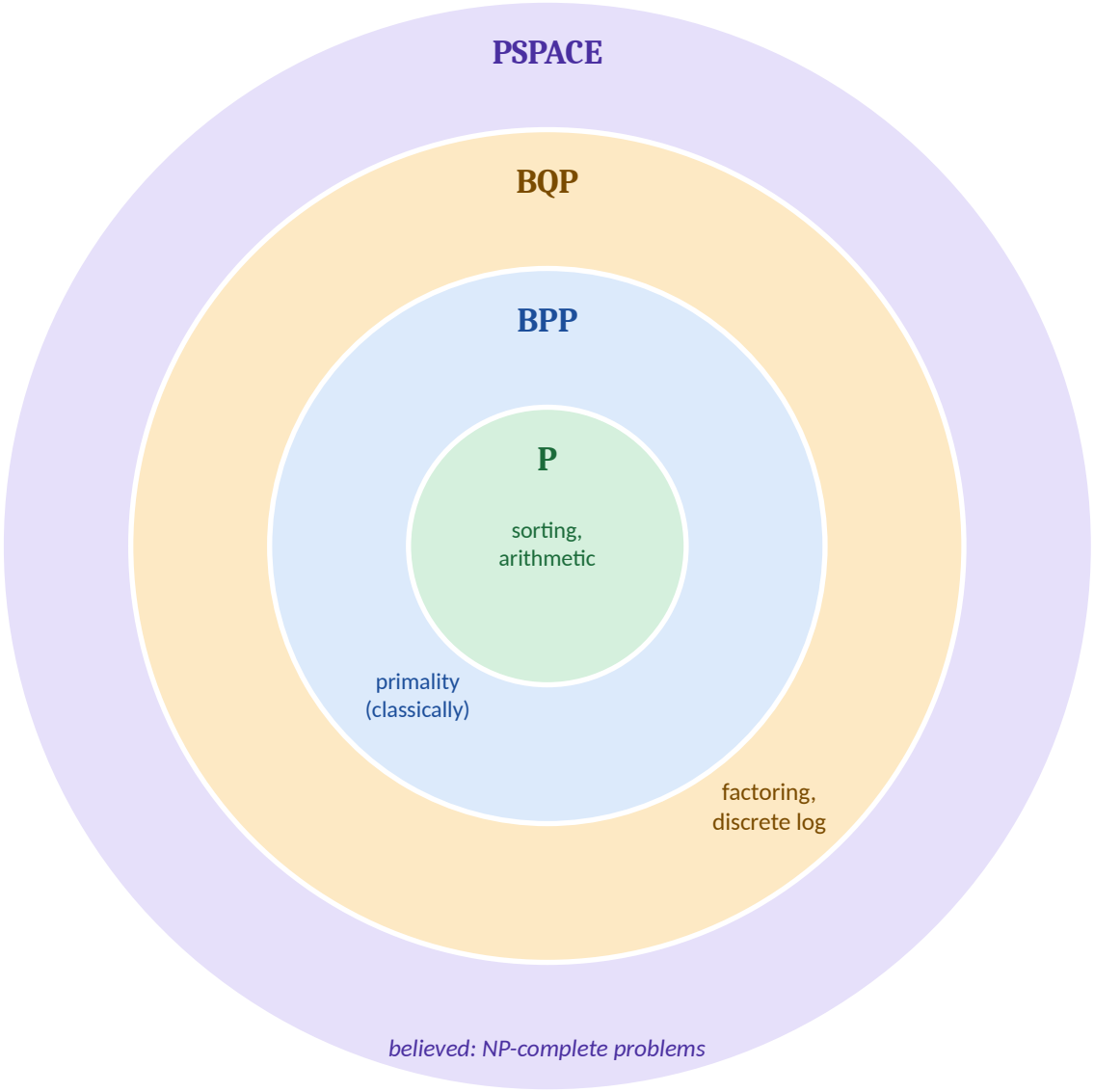
The framework

$$P \subseteq BPP \subseteq BQP \subseteq PSPACE$$

BPP is what classical computers solve efficiently (allowing randomness); BQP is what quantum computers solve efficiently.

Factoring sits in BQP but isn't known to be in BPP — that gap is exactly what Shor's algorithm exploits.

Note: BQP ⊆ PSPACE is proven, but whether BQP ⊆ P — i.e. whether quantum ever offers a real advantage — remains a famous open question.



4 Why √N is optimal: a quantum lower bound

Grover's algorithm isn't just good — it's provably the best possible.

The result

Any quantum search needs Ω(√N) oracle queries

The BBBV argument (Bennett–Bernstein–Brassard–Vazirani) shows that no quantum algorithm — Grover included — can find a marked item in fewer than order √N oracle calls, in the worst case.

So Grover's algorithm isn't just a good algorithm — it's optimal, up to a constant factor.

Note: lower bounds like this are rare — most quantum speedups are upper bounds only, with no proof that nothing faster exists.

Lower bound vs. what Grover achieves

N = 64	N = 1,024	N = 1,048,576
Lower bound Ω(√N)	Lower bound Ω(√N)	Lower bound Ω(√N)
~8	~32	~1,024
Grover achieves	Grover achieves	Grover achieves
6	25	804

Same order of magnitude at every scale — Grover doesn't just beat classical search, it meets the quantum limit.

5 Where quantum wins big: exponential speedups

A short list — every exponential speedup found so far exploits hidden structure.

Best known classical time vs. quantum time

Integer factoring	Discrete logarithm	Simulating quantum systems	Pell's equation / unit group
Classical	Classical	Classical	Classical
Sub-exponential (GNFS)	Sub-exponential	Exponential	Sub-exponential
Quantum	Quantum	Quantum	Quantum
$O((\log N)^3)$ — Shor	Polynomial — Shor's variant	Polynomial — Feynman / Lloyd	Polynomial — Hallgren

The pattern: every known exponential speedup exploits hidden algebraic structure — periodicity, group structure — not just brute-force search.

6 Where quantum doesn't help: no free speedup

Quantum computers are not a universal speedup machine.

The reality check

Most problems: quantum time ≈ classical time

For most computational problems, no quantum algorithm is known to beat the best classical one at all.

Even famous quantum-looking algorithms have caveats: HHL's speedup for linear systems needs sparse, well-conditioned matrices, and only returns a summary of the answer — not the full solution vector.

Careful: several early 'quantum machine learning' speedups were later matched by classical 'dequantized' algorithms of similar cost.

⇅

Sorted vs. unstructured data

Grover only helps for unstructured search; if data is already sorted, classical binary search wins outright.

NP

NP-complete problems

The best known quantum speedup here is only quadratic (via Grover) — no exponential speedup, and P vs NP stays open.

D

Dequantization

Classical algorithms have matched several proposed quantum machine-learning speedups once researchers looked for one.

7 The catch: overhead, noise, and NISQ limits

A speedup on paper still has to survive contact with real hardware.

Example: breaking RSA-2048 with Shor's algorithm

On paper: roughly 2,000 logical qubits and modest circuit depth — looks almost within reach.

With error correction: leading estimates put the requirement near 20 million noisy physical qubits, given today's error rates and surface-code overhead.

That's roughly 10,000× more physical qubits than logical ones — the overhead, not the algorithm, is today's bottleneck.

Physical vs. logical qubits

Every error-corrected logical qubit needs hundreds to thousands of noisy physical qubits today.

Gate fidelity & depth

Long algorithms need every gate to be extremely reliable — one stray error can spoil the whole run.

The NISQ gap

Today's ~100–1,000 qubit devices are noisy and uncorrected — far below cryptographic-scale Shor.

8 Why it matters: assessing real quantum advantage

A framework for asking whether a claimed speedup is real — not just a healthy dose of skepticism.

A framework for skepticism

Ask: proven? structured? worth the overhead?

Before believing a quantum speedup claim, check three things: is it mathematically proven or only observed on small cases? Does the problem have exploitable structure, like periodicity, or is it generic? And does the speedup survive once real hardware overhead is counted?

Note: ‘quantum advantage’ experiments (e.g. random circuit sampling) show a real computational gap — on a problem with no known practical use yet.

P

Proven vs. empirical

A mathematical proof, like BBBV, is stronger evidence than benchmarks on a handful of examples.

S

Structure matters

Every exponential speedup so far exploits deep algebraic structure — not a generic trick.

Σ

Count the overhead

A speedup that needs millions of extra physical qubits isn't an advantage yet — it's a roadmap.

ONE-LINE TAKEAWAY

Quantum speedups range from a modest, provably-optimal √N to an exponential leap — but every one has to survive real hardware overhead before it's a genuine advantage.

Proven lower bounds | Structure-dependent | Overhead-limited today

→

Next: Variational Quantum Algorithms (VQE & QAOA)

We turn from asymptotic promises to what today's noisy, error-uncorrected hardware can actually run right now.